



Virtualization Technology for Dependable Embedded Systems: A Survey for Transportation and Industrial Domains

JON PEREZ-CERROLAZA, Dependable Embedded Systems, Ikerlan Technological Research Centre, Arrasate-Mondragon, Spain and Basque Research and Technology Alliance (BRTA), Mendaro, Spain

PATRICIA BALBASTRE, Instituto AI2, Universitat Politècnica de València, Valencia, Spain

ENRICO MEZZETTI, Computer Science, Barcelona Supercomputing Center (BSC), Barcelona, Spain

ALFONS CRESPO, Dpto. de Informática de Sistemas y Computadores, Universitat Politècnica de València, Valencia, Spain

JON GONZÁLEZ EGUILUZ, Clue Technologies, Madrid, Spain

MIGUEL MARTIN ACOSTA, Airbus Defence and Space SA, Getafe, Spain

CHARLES-ALEXIS LEFEBVRE, Ikerlan Technological Research Centre, Arrasate-Mondragon, Spain and Basque Research and Technology Alliance (BRTA), Mendaro, Spain

Next-generation transportation and industrial systems are characterized by a continuously increasing number of integrated software functionalities and computational performance requirements. Nevertheless, there is also a need to reduce the cost, number, size, power consumption, volume and weight of integrated dependable embedded systems. In this context, virtualization technology enables the reconciliation of these challenges supporting the integration of multiple software functionalities of mixed-criticality (real-time and non-real-time, safety and non-safety-related) on a single computing device. This survey provides an overview of virtualization solutions (hypervisors, microkernels, and separation kernels) and categorizes research contributions addressing virtualization, real-time, dependability, and domain-specific technological challenges.

CCS Concepts: • **Software and its engineering** → **Embedded software; Real-time systems software; Computer systems organization** → **Real-time system architecture; Multicore architectures; Dependable and fault-tolerant systems and networks; Hardware** → *Safety critical systems;*

Additional Key Words and Phrases: Hypervisor, microkernel, separation kernel, virtualization, mixed-criticality

This work has been partially supported by European Research Council with Horizon 2020 (grant agreement No. 101082622), the Spanish Ministry of Science and Innovation (grant agreements No. PID2019-107255GB-C21, PID2021-124502OB-C41), and by grant PLEC2021-007609 funded by MICIU/AEI/10.13039/501100011033 and by the “European Union NextGenerationEU/PRTR”. The findings, conclusions and opinions expressed herein are solely those of the authors and do not necessarily represent the view(s) of affiliated institutions, organizations or funding bodies.

Authors’ Contact Information: Jon Perez-Cerrolaza, Dependable Embedded Systems, Ikerlan Technological Research Centre, Arrasate-Mondragon, Guipuzcoa, Spain and Basque Research and Technology Alliance (BRTA), Mendaro, Guipuzcoa, Spain; e-mail: jmperez@ikerlan.es; Patricia Balbastre, Instituto AI2, Universitat Politècnica de València, Valencia, Comunitat Valenciana, Spain; e-mail: patricia@ai2.upv.es; Enrico Mezzetti, Computer Science, Barcelona Supercomputing Center (BSC), Barcelona, Barcelona, Spain; e-mail: enrico.mezzetti@bsc.es; Alfons Crespo, Dpto. de Informática de Sistemas y Computadores, Universitat Politècnica de València, Valencia, Comunitat Valenciana, Spain; e-mail: acrespo@ai2.upv.es; Jon González Eguiluz, Clue Technologies, Madrid, Madrid, Spain; e-mail: jon.gonzalez@clue.aero; Miguel Martin Acosta, Airbus Defence and Space SA, Getafe, Madrid, Spain; e-mail: miguel.martin.acosta@airbus.com; Charles-Alexis Lefebvre, Ikerlan Technological Research Centre, Arrasate-Mondragon, Pais Vasco, Spain and Basque Research and Technology Alliance (BRTA), Mendaro, Guipuzcoa, Spain; e-mail: calefebvre@ikerlan.es.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2025 Copyright held by the owner/author(s).

ACM 0360-0300/2025/11-ART108

<https://doi.org/10.1145/3765736>

ACM Reference Format:

Jon Perez-Cerrolaza, Patricia Balbastre, Enrico Mezzetti, Alfons Crespo, Jon González Eguiluz, Miguel Martin Acosta, and Charles-Alexis Lefebvre. 2025. Virtualization Technology for Dependable Embedded Systems: A Survey for Transportation and Industrial Domains. *ACM Comput. Surv.* 58, 4, Article 108 (November 2025), 37 pages. <https://doi.org/10.1145/3765736>

1 Introduction

In recent decades, transportation and industrial domain systems (e.g., automotive [110, 171, 211], **Industrial Control Systems (ICSs)** [82], railway signaling [13, 37], space satellites [152, 232], wind turbines [167, 168]) have integrated increasingly complex functionalities following a federated architecture approach where each major functionality is provided by a dedicated embedded system [85]. This approach has led to a greater number of embedded systems, wires and connectors, which limits its scalability. For instance, “premium cars have more than 20 million **Lines of Code (LoC)** deployed in” over 150 **Electronic Control Units (ECUs)** [171, 211], driving up electronics costs from 20% to 35% of the total car cost between 2000 and 2017 [110]. Furthermore, next-generation transportation and industrial systems, such as **Advanced Driver-Assistance Systems (ADAS)** and **Autonomous Driving (AD)** systems that integrate **Machine Learning (ML)** algorithms, process vast amounts of data and require the integration of complex software stacks, will further amplify this scalability limitation. All in all, there is a common need to facilitate the continuous integration of more functionalities while reducing the overall cost, number, **Size, Weight and Power (SWaP)** of integrated embedded systems [110, 148, 154, 169–171, 201]. In this scenario, an integrated architecture approach, where multiple mixed-criticality functionalities (real-time and non-real-time, safety and non-safety-related) are integrated into a single embedded system, could provide the required scalability [85].

Virtualization technology was initially developed in the '60s to partition mainframe computer systems, and nearly 50 years ago, Popek and Goldberg [179] established a set of requirements for efficient virtualization such as trapping privileged instructions and enabling the emulation of the intended effect in the guest OSs [222]. Later, virtualization technology embraced the PC market in the '90s, and nowadays further extended its reach to dependable embedded systems. At the heart of virtualization technology is the **Virtual Machine Monitor (VMM)** (a.k.a., hypervisor), a software layer that enables the creation of **Virtual Machines (VMs)** and execution of *software partitions*, where a physical computing machine (a.k.a., *host*) is partitioned into multiple VMs that provide the illusion of being host replicas (a.k.a., *guests*). The VM abstraction includes the **Virtual CPUs (vCPUs)**, resources (e.g., memories, timers, interrupts) and peripherals [85, 179, 192]. When designing a virtualization solution, there are multiple approaches to consider, and the computing device itself places significant constraints on the design, features and overall solution. Furthermore, the efficient virtualization, with **Temporal and Spatial Partitioning (TSP)** of shared peripherals (e.g., memory, caches, communication peripherals, I/Os) is also a research and technical challenge [169, 171, 222]. Following the analogy of mainframe computers and PCs, embedded processors and the latest embedded **High-Performance Computing (HPC)** devices (e.g., **Multi-Processor System-on-a-Chips (MPSoCs)** [171], **Graphics Processing Units (GPUs)** [169]) can provide the necessary computing performance to continue integrating more functionalities. And virtualization technology can support the integration of mixed-criticality software partitions on a single computing device. However, the features and requirements of dependable embedded systems (e.g., resource constraints) and applications targeting transportation and industrial domains, differ significantly from those of servers, cloud computing and mobile devices. Therefore, some virtualization solutions were specifically designed for dependable embedded systems (e.g., PikeOS, XtratuM), while others have evolved from server-oriented and mobile domains (e.g., Xen, KVM).

Dependability attributes (i.e., reliability, availability, maintainability, safety, integrity [21]) are common requirements in industrial and transportation systems. Moreover, dependability attributes are becoming increasingly relevant also in other domains, such as mobile devices, cloud computing and data centers, where virtualization solutions play a crucial role [85, 104, 192, 202, 212]. As a result, virtualization technology for dependable systems is an active area of research, with hundreds of research contributions. For instance, a title, abstract and keyword-based search in Scopus for virtualization technology and dependability (or associated attributes), restricted to computer science, yields over 3,500 publication results. And even when excluding domains outside the scope of the survey (e.g., *mobile*, *cloud*, *data-center*), there are still over 1,000 research publications.¹

The survey is organized as follows. First, Section 2 introduces the basic concepts and terms used in this work. Next, Section 3 summarizes virtualization solutions for dependable embedded systems (hypervisors, microkernels and separation kernels). Then, Sections 4, 5, 6, and 7 describe research contributions addressing virtualization (Section 4), real-time (Section 5), dependability (Section 6) and domain-specific (Section 7) challenges. On top of that, Table 1, summarizes the categorization of the most relevant related works considered in the previous Sections. Finally, Section 8 provides the overall conclusion and future research directions. Additionally, this survey references other virtualization technology surveys/reviews that provide complementary or more detailed content for a given topic or application domain: concepts and taxonomy [192, 197], formal verification [245], scheduling [81], embedded systems [212, 237], safety-critical embedded systems [85, 131] and other application domains (cloud computing [104], mobile [202]).

2 Background

This section summarizes basic concepts and terms used within the survey.

2.1 Dependability

This survey builds on the dependable computing terminology introduced by Avizienis et al. [21] that defines dependability as “the ability to deliver service that can justifiably be trusted” and encompasses five attributes: *reliability* (“continuity of correct service”), *availability* (“readiness for correct service”), *maintainability* (“ability to undergo modifications and repairs”), *safety* (“absence of catastrophic consequences”) and *integrity* (“absence of improper system alterations”). In avionics, the term integrity usually refers to a measurement of the accuracy and correctness of the calculations and data of the system. But in the survey, the term integrity is used as generally defined in [21].

Safety-critical embedded systems are programmable electronic systems that protect or provide a critical service. If they fail, they can cause harm or damage (e.g., railway signaling accident). In some cases, these systems must undergo strict certification processes to minimize, to extremely low levels, the probability of failure due to systematic errors (e.g., human, tool error) and random hardware errors (e.g., memory bit flip), in compliance with generic (IEC 61508 [93]) or domain-specific standards such as ISO 26262 [96] (automotive), ARP4754 and DO-178C [188] (avionics), EN 50126/8/9 (railway), ECSS-Q-ST-30C/40C [67] (space) and ISO 10218-1 (robotics). Due to the variability of terms, definitions and requirements among previously listed safety standards, the survey uses the generic IEC 61508 as the reference safety standard [169, 171]. More information about safety-critical systems and safety standards can be found in [85, 169, 171].

¹(TITLE-ABS-KEY(virtualization) OR TITLE-ABS-KEY(hypervisor) OR TITLE-ABS-KEY(microkernel)) AND (TITLE-ABS-KEY(dependability) OR TITLE-ABS-KEY(reliability) OR TITLE-ABS-KEY(availability) OR TITLE-ABS-KEY(maintainability) OR TITLE-ABS-KEY(safety)) AND NOT TITLE-ABS-KEY (mobile) AND NOT TITLE-ABS-KEY (cloud) AND NOT TITLE-ABS-KEY (datacenter) AND NOT TITLE-ABS-KEY (data-center) AND NOT TITLE-ABS-KEY (medical) AND NOT TITLE-ABS-KEY (“network virtualization”) AND NOT TITLE-ABS-KEY (SDN) AND NOT TITLE-ABS-KEY (server) AND NOT TITLE-ABS-KEY (VoIP) AND NOT TITLE-ABS-KEY (proceedings) AND (LIMIT-TO (SUBJAREA,“COMP”)).

Table 1. Categorization of the Related Works

Domain		Domain-specific case-studies and challenges	Sect.
Transportation	Automotive	· <i>Functional safety</i>: E-gas [36, 154] · <i>Virtualization and real-time evaluation</i>: AD [32, 36, 43, 83], cockpit/infotainment/HMI [39, 110, 178], generic [43, 63, 83, 183, 186, 187, 206, 211]	Section 7
	Avionics	· <i>Functional safety</i>: Generic approach [224, 227] · <i>Virtualization and real-time evaluation</i>: Generic [226], multicopter [75, 184], UAS [194]	
	Railway	· <i>Functional safety</i>: Railway signalling SIL4 [13, 37]	
	Space	· <i>Virtualization and real-time evaluation</i>: Benchmark [152], overhead analysis [105, 112, 166], fault-tolerance [45, 112], reference architecture [80]	
	ICS	· <i>Virtualization and real-time evaluation</i>: Machine control case-study [25]	
Industrial		· <i>Functional safety</i>: Safety protection SIL3/Pld [120, 167, 168]	
Technical Challenge		Contribution(s)	Sect.
Dependability	Integrity	· <i>Cybersecurity/Security</i> [113, 150, 226, 231, 242] · <i>Execution Integrity</i> [134, 167, 168, 224]	Section 6
	Safety	· <i>Diagnosis and system reactions to errors</i> [134, 167–169, 171, 181, 222] (see also <i>Reliability</i>) · <i>Safe IPC</i> [85, 110, 122, 236] (see also <i>Virtualization - services for partitions</i>) · <i>Spatial independence</i> [57, 87, 93, 122, 167–169, 171, 181, 187, 211, 222, 233] · <i>Temporal independence</i> [38, 40, 41, 57, 116, 122, 137, 167–169, 171, 181, 218, 224] (see also <i>Real-Time (RT)</i>) · <i>Safe startup/shutdown</i> [42, 110, 123, 168, 184, 211] · <i>Resource virtualization and exclusive access to peripherals</i>: [57, 93, 110, 111, 167, 168, 184, 222, 224] (see also <i>Virtualization</i>) · <i>Safety compliance</i> [13, 26, 36, 38, 40, 53, 57, 69, 80, 82, 84, 85, 91, 93, 106, 110, 114, 118, 121–123, 131, 138, 144, 151, 167, 168, 181, 184, 196, 222, 224, 226, 227, 242]	
	Maintainability	· <i>Maintainability</i> [92, 124, 185, 187]	
	Availability	· <i>Run-time partition migration</i> [134, 192] · <i>Backup and recovery</i> [132] · <i>Dynamic system reconfiguration</i> [97]	
	Reliability	· <i>Random errors management</i> [31, 54, 57, 69, 71, 72, 85, 144, 167–169, 171, 193, 200, 222] · <i>Systematic errors management</i> [14, 24, 26, 27, 34, 35, 42, 57, 74, 90, 114, 125, 214, 224, 243, 245]	
	Temporal requirements (soft, firm, hard)	· <i>Analysis and characterization of virtualization overheads on execution time</i> [12, 33, 50, 56, 97, 100, 101, 112, 115, 116, 141, 147, 149, 152, 153, 187, 220, 233, 244] · <i>Interrupts temporal interference management</i> [29, 40, 57, 148, 223, 233] · <i>HPC devices temporal interference management</i> [3, 17–19, 29, 40, 49, 64, 66, 94, 115, 116, 135, 145, 157, 158, 160, 161, 169, 171, 198, 223, 228, 239, 246] · <i>WCET analysis</i> [11, 13, 120, 169, 171] · <i>Partition scheduling</i> [59, 81, 85, 107, 136, 173, 240, 241, 247]	
	Virtualization and resource management	· <i>Services for partitions</i> [38, 57, 85, 162, 180] · <i>Hardware observability and resources virtualization</i> [65, 77, 95, 130, 142, 155, 156, 159, 208, 209, 217, 229, 238] · <i>Peripheral virtualization</i> [34, 38, 41, 41, 49, 57, 99, 149, 153, 169, 171, 182, 183, 187, 222] · <i>CPU and associated resources virtualization</i> [48, 169, 171, 174, 183, 186, 187, 211, 222]	
Virtualization Type		Virtualization Solution(s)	Sect.
Hypervisor	Type 1	· AIR [79] · Bao [137, 190] · BlueVisor [100, 102, 103] · CLARE [38, 41] · COQOS [4, 76] · Hyper-V [143] · LTZVisor [162, 172–174, 176] · LWHVR [6, 63, 183, 186, 187, 211] · Proteus [78] · QNX hypervisor [178] · Rodosvisor [48, 215, 216] · RTS hypervisor [7] · RTZvisor [175] · Shyper [199] · Xen [12, 15, 23, 33, 36, 38, 41, 51, 61, 68, 81, 97, 110, 141, 155, 165, 194, 207, 223, 225, 227, 233, 238–241, 244] · XtratuM/XNG [37, 45–47, 55–58, 80, 112, 120, 138–140, 151, 152, 166–168, 181, 218, 219, 221, 222, 232] · XVisor [38, 145, 165] · Virtuosity [225] · VOSYSmonitor [44, 133, 134] · Wind River hypervisor (Helix platform) [236] · μRTZVisor [136] · μ-visor [8]	Section 3
	Type 2	· Jailhouse [22, 32, 38, 39, 43, 53, 61, 184, 203, 244, 246] · KVM [5, 12, 15, 32, 38, 43, 59, 60, 68, 81, 85, 165, 207, 247]	
Microkernel		· EB Corbos/L4Re [9, 117, 118] · PikeOS [2, 13, 26, 27, 29, 30, 66, 83, 107, 108, 152, 157, 158, 228] · seL4 [31, 40, 42, 88, 90, 109, 114, 125, 132, 224, 226]	
Separation Kernel		· LynxSecure [135, 217, 227] · Multivisor [8, 113] · Quest-V [62, 128, 205, 233] · VXWorks 653 [163]	

The development of dependable embedded systems must consider real-time requirements, dependability attributes and domain-specific challenges, which are often interconnected. These systems must meet real-time requirements to provide a service that can be justifiably trusted (provide the intended function(s) when needed). The criticality of real-time requirements (e.g., soft/hard real-time) and the dependability attributes impose technical constraints, requirements and challenges in the system development and selection of virtualization solutions. For instance, safety-critical systems must meet hard real-time requirements, provide diagnostics and comply with applicable safety standards. Additionally, each domain presents unique challenges. For example, some aerospace systems must withstand harsh operating conditions (reliability), and adhere to domain-specific execution environments (e.g., ARINC 653) and stringent safety standards.

2.2 Multi-Core/Many-Core Computing Devices

The increasing adoption of complex heterogeneous HPC devices (e.g., MPSoCs, GPUs) in dependable embedded systems leads to spatial/temporal independence, reliability and safety certification challenges [169, 171]. *Spatial independence* is necessary to prevent one element from inappropriately altering data used by another element. Design constraints such as static and exclusive access to resources, and the usage of **Memory Management Unit (MMU)** hardware components are common methods to support spatial independence. Meanwhile, *temporal independence* is necessary to ensure that one element does not cause another to malfunction by monopolizing too much of the available CPU execution time. Software applications deployed in HPC devices designed for maximum average performance are subject to multiple sources of temporal interference (e.g., shared resources) [169, 171]. This can lead to high variability in execution time [28], high difficulty (if feasible) to provide **Worst Case Execution Time (WCET)** estimates with a reasonable effort [66, 171, 228, 234], and ultimately complicates devising trustworthy time bounds for ensuring that time constraints and time budgets are always met [10, 170, 171]. In the avionics domain, spatial and temporal independence are part of the *robust partitioning* concept [1]. Finally, techniques such as diagnosis and fault-tolerance aim at increasing the overall computing device *reliability* by detecting-reacting and tolerating random hardware errors [169, 171].

2.3 Virtualization Technology

The state-of-the-art virtualization technology is the result of an evolution process that started in the '60s and the convergence of heterogeneous technologies such as **Operating Systems (OSs)**, HPC devices, compilers, and interpreters [85]. This has caused the overloading of concepts, terms and taxonomies [85, 98, 192, 197], ultimately resulting in a “terminological confusion” [85]. This subsection introduces the main concepts and terms for the scope of the survey.

2.3.1 Core Technologies for Virtualization. In a conceptually similar way to OSs that support abstracting and managing hardware computing devices to be used by software applications under a suitable interface, virtualization solutions support abstracting and offering the physical components of a computing device (a.k.a., computer, machine) to execution environments in the form of so-called VMs using a software layer (hypervisor). However, in recent years, the term hypervisor has been extended and used in a way that can lead to misunderstandings in its interpretation. In this survey, the term *hypervisor* refers to a software layer that abstracts and offers computing device resources (e.g., CPU), properties (e.g., spatial isolation) and services to execution environments as if they were running on a dedicated native machine (VM).

Furthermore, as depicted in Figure 1, hypervisors can be complemented by other solutions that offer similar support, such as *microkernels* and *separation kernels* (SKs). In the 1980s, there was a trend to isolate the basic components of the OS from the higher-level parts integrated into the

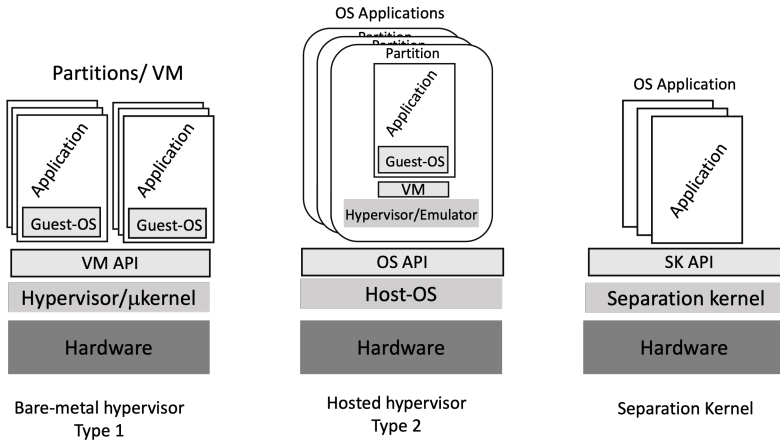


Fig. 1. Core technologies for virtualization: Hypervisor (Type 1 and 2), microkernel and separation kernel.

monolithic OS design. This led to the emergence of microkernels, where the core of the OS manages the processor, memory and interrupts, and leaves other services to higher layers by implementing them as servers. It is important to note that there is a conceptual difference between a hypervisor as a VM facilitator and the microkernel interface oriented to user processes. While they are conceptually different, when an application with its OS is placed in a microkernel user process, the virtualization vision tends to resemble each other, and, therefore, many solutions based on microkernels are defined as hypervisors. Additionally, to reduce and improve the management of failures, Rushby [189] proposed the *separation kernel* design approach for OSs reinforcing the temporal and spatial isolation of applications. As part of this approach, an extended interface to handle the **application execution environments (APEX)** was defined (e.g., ARINC 653 standard [16] in the avionics domain).

Another view of virtualization is to build applications (e.g., emulators) that offer (emulate or simulate) the behavior of a given hardware. These are applications that run on top of an OS and can provide a VM similar to the native or can emulate different hardware. However, due to a low execution overhead and near-native hardware performance, previous virtualization solutions are the most common ones for dependable embedded systems, while virtualization applications such as emulators are notably less common [20]. Therefore, virtualization applications such as emulators are considered outside the scope of this survey.

Finally, hypervisors can be classified as *Type 1 or 2*. A *Type 1* hypervisor (a.k.a., bare-metal) runs directly on the bare-metal/native hardware. And a *Type 2* hypervisor (a.k.a., hosted) runs inside or on top of a host OS, either as an additional application/service (e.g., emulator) or integrated with the OS kernel (e.g., microkernel). Nonetheless, some virtualization solutions might not fall into a single category as they might share features of different categories. For instance, **Kernel-based Virtual Machine (KVM)** can be classified as a Type 2 hypervisor because they are installed as Linux modules and loaded after the OS performs the initialization, and as a Type 1 hypervisor because they take direct control of the underlying native hardware once loaded [85].

2.3.2 Type of Virtualization. A hypervisor can provide two main types of *virtual environments*, classified as *full virtualization* and *paravirtualization*. While *full virtualization* allows running an unmodified guest OS on top of the hypervisor, *paravirtualization* requires adapting the guest OS to use the hypervisor services instead of accessing the hardware directly. Additionally, two basic techniques can be used to support *full virtualization*: *code analysis with associated dynamic binary translations* and *hardware-assisted virtualization* [85, 192].

2.3.3 Software Partition. A *software partition* (or *partition* for short) “consists of one or more concurrent tasks or processes (implemented by the OS within each partition), sharing access to processor resources” [189]. Multiple partitions can execute on a computing device, where the virtualization solution can provide several vCPUs to a partition, and a vCPU can (sequentially) execute multiple partitions. *Normal partitions* can only use general services offered by the virtualization solution, and *supervisor partitions* (a.k.a., *master partitions*) can additionally use special services to manage partitions (e.g., stop a partition) and other privileged services [57, 85].

3 Virtualization Solutions

Table 2 provides a systematic listing of virtualization solutions (hypervisors, microkernels and separation kernels) referenced in research publications and targeting the development of dependable embedded systems. This Section also provides a brief description of selected virtualization solutions. While there are many virtualization solutions available, only a relatively small number of them have been widely covered in scientific publications (e.g., Jailhouse, PikeOS, seL4, Xen, XtratuM/XNG). Multiple solutions have only a few references and others are well-known commercial solutions with limited publicly available and peer-reviewed research contributions (e.g., LynxSecure, VxWorks).

Virtualization solutions typically exhibit a small code footprint, with sizes that can range from approximately ten thousand to a few hundred thousand LoCs (e.g., seL4 9,300 [114], KVM 220,000 [68], Xen 300,000 [68] LoC). And due to performance reasons, the most common software implementation language is C, with some minor hardware-specific implementations in assembly language [26, 114] (e.g., seL4: 8,700 LoC in C and 600 LoC in assembly [114]).

Supported computing architectures are mainly limited to the most common computing architectures (ARM, PowerPC, x86), domain-specialized computing architectures (e.g., LEON for space, TriCore for automotive) and new computing architectures with considerable research activity (e.g., RISC-V) [169, 171]. Regarding supported OSs, there is a higher variability but the most common **General Purpose Operating System (GPOS)** is (possibly custom distributions of) Linux [131] and the most common **Real-Time Operating System (RTOS)** is FreeRTOS. Instead, the supported *execution environments* (e.g., AUTOSAR, ARINC 653) and *safety integrity levels* are strongly determined by the domain-specific origin or target domains of a given virtualization solution, as we will further discuss in Section 7.

Creating a detailed guideline for selecting and evaluating virtualization solutions is beyond the given survey’s scope and should also be tailored to domain-specific requirements (e.g., automotive platform [83]). As a general recommendation, consider the need for an application-specific thorough analysis of required real-time properties and guarantees, domain-specific requirements (e.g., execution environments), safety compliance requirements, legacy requirements, computing device resources and characteristics (e.g., hardware virtualization support) and required OS support. Also, consider performing a thorough analysis (pros and cons) of preselected virtualization solutions constraints (e.g., paravirtualization requires adapting the guest OS), supported and provided features (e.g., communication services, schedulers, multi-core device, security), overheads (e.g., memory footprint, boot time overhead, context-switch overhead), required software engineering effort (e.g., drivers development), industrial maturity and business model [83]. Furthermore, as a rule of thumb, consider that *Type 1* hypervisors tend to have the smallest code size and temporal overhead; other virtualization solutions (not *Type 1*) are based on a given OS (e.g., Linux), providing a richer set of services; and that separation kernels provide standardized interfaces for partitions (e.g., ARINC 653) while other solutions commonly provide proprietary and diverse interfaces.

3.1 Hypervisor—Type 1

This Section briefly describes selected *Type 1 hypervisors* (LTZVisor/RTZVisor/ μ RTZVisor, Xen and XtratuM/XNG). Furthermore, Table 2 provides a comprehensive summary of all relevant

Table 2. Summary of Virtualization Technology Solutions

Type		Virtualization Solution		Architec- ture(s)	Operating System(s)	Supported Execution Environment(s)	Safety Integrity Level	Scheduler(s)
Hypervisor	Type 1	AIR	Para	ARM, SPARC	RTEMS	ARINC 653	-	Cyclic
		Bao	Full	ARM, RISC-V	Erika, FreeRTOS, Linux, Android	-	-	Static
		BlueVisor	Para	ARM, MicroBlaze	FreeRTOS, uC/OS-II, Xikernel	-	-	Dynamic
		CLARE	Full	ARM	Linux, FreeRTOS, Erika, ROS2	AUTOSAR	-	Static
		COQOS	Full	ARM	Linux	AUTOSAR	Automotive ASILB	(Not specified)
		LTZVisor	Full	ARM	FreeRTOS, Linux	-	-	Priority based
		LWHVR	Para	PowerPC, Tricore	ETAS RTA-OS	AUTOSAR	Automotive ASILB	Cyclic
		Hyper-V	Full	x86	Windows, Linux, FreeBSD	-	-	Round-robin
		Proteus	Full & Para	PowerPC	-	-	-	Cyclic
		QNX Hypervisor	Full	ARM, x86	QNX Neutrino, QNX OS, Linux, Android	AUTOSAR	Automotive ASILD Industrial SIL3	Priority based
		Rodosvisor	Full & Para	PowerPC	-	ARINC 653 (not all services)	-	Cyclic
		Real-Time Hypervisor (RTS)	Full	x86	Windows, Linux, VxWorks, Xenomai, QNX, T-kernel	-	-	(Not specified)
		RTZVisor	Full	ARM	RODOS OS	-	-	Static
		Shyper	(Not specified)	ARM	FreeRTOS, Linux	-	-	Static, priority, round-robin
		VOSYSmonitor	Full	ARM, RISC-V	FreeRTOS, Linux	-	Automotive ASILC	Priority based
		Xen	Para	ARM, x86	Erika, Linux, FreeBSD, NetBSD, Solaris, Android, FreeRTOS	-	-	Cyclic, Priority based, Server based
		XtratuM/XNG	Para	ARM, LEON, RISC-V	Linux, LithOS, RTEMS	ARINC 653	Space Level B	Cyclic
		XVisor	Full & Para	ARM, x86, RISC-V	Linux, etc.	-	-	Cyclic
		Wind River hypervisor (Helix platform)	Para	ARM, x86	Wind River Linux, VxWorks, Windows	-	Automotive ASILD Avionics DAL A Industrial SIL3	Static, priority
		μRTZVisor	Full	ARM	FreeRTOS	ARINC 653	-	Server + priority
		μ-visor	(Not specified)	ARM, RH850/U2A, TriCore	FreeRTOS, μ-velOSity	AUTOSAR	Automotive ASILD Industrial SIL3	Static or priority
	Type 2	Jailhouse	Para	ARM, x86	Linux, Android, FreeRTOS, Erika, Zephyr	AUTOSAR	-	Static
		KVM	Full	ARM, PowerPC, x86, RISC-V	Linux, Android, FreeRTOS, FreeBSD, NetBSD, Solaris, Windows	-	-	Priority based, server based, round-robin
Microkernel		EB Corbos / L4Re	-	ARM, x86	QNX, Linux	AUTOSAR	Automotive ASILD	Fixed priority
		PikeOS	-	ARM, x86, LEON, MIPS, PowerPC	Linux, POSIX PSE51, ITRON, OSEK OS	ARINC 653, Ada RTS, JVM, soft-PLC runtime system	Automotive ASILD Avionics DAL A Industrial SIL3 Railway SIL4	Cyclic + priority
		seL4	-	ARM, x86, RISC-V	Linux	-	-	Priority based
Separation kernel		LynxSecure	-	ARM, PowerPC, x86	FreeRTOS, Linux, LynxOS, Windows	ARINC 653	(Support for Avionics)	Cyclic
		Multivisor	-	ARM, x86	Integrity RTOS, Linux, Android, Windows	AUTOSAR	Automotive ASILD Avionics DAL A Railway SIL4 Industrial SIL3	Static or dynamic
		Quest-V	-	x86	Quest, Linux, Android	AUTOSAR	-	Server based
		VXWorks 653	-	ARM, Intel PowerPC	Wind River Linux, VxWorks Cert	ARINC 653	Automotive ASILD Avionics DAL A Industrial SIL3	Cyclic

virtualization solutions for the scope of the survey: AIR [79], Bao [137, 190], BlueVisor [102, 103], CLARE [38, 41], COQOS [4, 76], Hyper-V [143], LHWHR/RTA HV [6, 187], Proteus [78], Rodosvisor [48, 215, 216], QNX Hypervisor [178], **Real-Time Hypervisor (RTS)** [7], Shyper [199], XVisor [145, 165], Virtuosity [225], VOSYSmonitor [44, 133], Wind River hypervisor (Helix platform) [236] and μ -visor [8]. Other *Type 1* hypervisor solutions have been excluded because their target domains fall outside the scope of this survey (e.g., OKL4 targets mobile devices [89], Hafnium is oriented towards the separation between security-critical and untrusted code [119]).

3.1.1 LTZVisor, RTZVisor and μ RTZVisor. LTZVisor is an open-source and ARM TrustZone-based hypervisor [162, 172, 174, 176] with support for a dual-OS configuration (FreeRTOS, Linux). Based on this hypervisor, the same research team has developed specialized variants such as RTZVisor that targets space applications [174, 175], and μ RTZVisor “a refactoring of RTZVisor to achieve a higher degree of safety and security” with mono-core ARM processors [136, 173].

3.1.2 Xen. Xen [23, 51] is a paravirtualized hypervisor that supports a wide range of applications, from servers and desktops to dependable embedded systems. In Xen, hypervisor partitions are called *domains*. The *Domain 0* or *dom0* is the first to run when Xen boots. The guest OS that runs in *dom0* is often Linux but can be others such as NetBSD, Solaris or FreeBSD. The rest of the domains are referred to as *domain U* or *domU* and they are unprivileged domains in contrast to *dom0*. There is an ongoing project to certify Xen for ISO 26262 ASIL B. The first steps to achieve this certification are to use an RTOS as *dom0* and MISRA C compliance. But, at this time, there is no known version of Xen that meets this functional safety standard. The Xen porting to the Xilinx Zynq UltraScale+ Multi-Processor on a Chip (MPSoC) is called *Virtuosity* hypervisor [225].

3.1.3 XtratuM/XNG. XtratuM is a bare-metal hypervisor that provides extended capabilities for developing high-criticality systems and “provides a raw (close to the native hardware) virtual execution environment, rather than a full-featured one”. XtratuM supports x86, LEON2/3/4 and ARM architectures [46, 58, 139, 140]. The latest version of XtratuM is called XNG and it is currently qualified to ECSS level B to run in LEON3 and ARM Cortex R4/R5 processors (single core), and in Zynq7000 SoCs in multicore configuration. XtratuM supports XRE (minimal run-time for bare-metal applications), LithOS (ARINC 653 compliant RTOS), RTEMS and Linux as guest OSs.

3.2 Hypervisor—Type 2

The number of *Type 2* hypervisors considered in dependable systems for transportation and industrial domains is smaller, with two relatively more popular solutions (Jailhouse and KVM). As summarized in Table 2, there are fewer Type 2 hypervisors than Type 1 hypervisors. Type 2 hypervisors run as applications of an existing host GPOS, such as Linux, which provides the underlying computing foundation. Their use is commonly limited to soft real-time applications due to WCET and real-time analyzability challenges arising from the complexity of the host GPOS and the entire execution chain (application, guest OS, hypervisor, host GPOS and hardware). In contrast, Type 1 hypervisors run bare-metal on diverse hardware, offering efficient resource management with minimal computing overhead, and supporting hard and soft real-time applications. This results in a broader range of diverse challenges, specialized strategies and Type 1 solutions.

3.2.1 Jailhouse. As a *Type 2* hypervisor, Jailhouse [203] needs an OS to boot (Linux). Unlike other hypervisors such as KVM or Xen, it was designed with simplicity as a primary goal. A Jailhouse-based system organizes partitions into cells. There is always at least one cell known as the Linux cell, similar to *dom0* in Xen. This cell contains the Linux system that launches the hypervisor and serves to control it afterwards. Then, it can run bare-metal applications or ported

OSs. The Jailhouse design builds on static resource configuration, which means, for example, that cells cannot share cores and resource assignment is static.

3.2.2 KVM. KVM [5, 85] is a solution to implement full virtualization in Linux, already integrated with the Linux kernel (since release 2.6.20). As a *Type 2* hypervisor, KVM is loaded as a module of the Linux kernel. KVM provides an interface to user space, based on which a normal process can request to be transformed into a virtual machine. Normally, I/O devices are emulated (usually using QEMU) in user space while KVM takes care of virtual CPUs and memory management. Finally, KVM uses existing functionalities in the Linux kernel (e.g., memory management, scheduler), making KVM code smaller than other *Type 2* hypervisors.

3.3 Microkernel

Concerning *microkernels*, this section briefly describes L4 family and PikeOS solutions.

3.3.1 L4 Microkernel Family. The L4 microkernel [129] is a small and highly efficient OS kernel designed to provide a minimal set of functions in embedded systems. It is based on the concept of microkernel architecture which involves breaking down the OS into small, independent parts that run in their own protected space. The L4 microkernel has been used in a variety of systems, including real-time systems and mobile devices (e.g., OKL4). Some of the variants of L4 target dependable embedded systems. For instance, seL4 is the first microkernel to be formally verified [31, 88, 109, 114, 132]. Additionally, EB Corbos / L4Re is an operating system based on the L4 microkernel, designed to provide a complete platform for building and running system-level software [117]. The commercial version of L4Re is EB Corbos [9, 118], which supports AUTOSAR and it is safety certified (ISO 26262 ASILD).

3.3.2 PikeOS. PikeOS is a microkernel targeting safety-critical applications [108]. It was initially designed as a solution to certain issues identified in the L4 kernel interface with respect to temporal and spatial isolation. PikeOS has the following relevant features: all resource allocation is assigned during system startup by means of a static configuration table; usage of processor privilege levels to protect the system from user domains; memory and I/O is partitioned with MMU; supports multiple pre-defined scheduling plans; inter-domain communication and with I/O devices follows ARINC 653 standard; supports application-level error handling.

3.4 Separation Kernel

Regarding *separation kernels*, this section briefly describes the Quest-V solution. In addition, there are additional **Commercial Off-The-Shelf (COTS)** solutions with limited (public and peer-reviewed) research references (e.g., LynxSecure [135, 227], Multivisor [8, 113], VXWorks 653 [163]).

3.4.1 Quest-V. Quest-V is the virtualized version of Quest RTOS [233]. In Quest-V domains/partitions are called sandboxes and isolation is supported by hardware-assisted virtualization techniques. A sandbox is “a hardware partition that hosts a well-defined software environment” [128]. Each sandbox operates with its own guest OS (paravirtualized) and is granted exclusive access to a specific subset of hardware resources of the computing device (e.g., CPUs, memory regions and I/O). Quest-V employs the Quest RTOS to allocate at boot time these hardware resources, which are afterward directly handled within the kernels/user-spaces of each sandbox (they are not virtualized).

4 Virtualization Challenges

This Section provides an overview of the primary challenges associated with *virtualization*, which are expanded upon in subsequent sections covering real-time (Section 5), dependability (Section 6) and domain-specific (Section 7) challenges. Virtualization solutions must support the integration of

multiple VMs and software partitions of mixed-criticality, virtualizing and encapsulating computing resources both temporarily (e.g., execution slot) and spatially (e.g., preventing the unintended alteration of memory/code among partitions) with a reduced and predictable computing overhead and a small footprint. For that purpose, the virtualization solution must at least provide CPU and associated resources virtualization (Section 4.1), peripheral (Section 4.2) and specialized resources (Section 4.3) virtualization, and services to partitions (Section 4.4). All in all, supporting the required temporal and spatial partitioning among partitions using spatial/temporal interference avoidance, mitigation, detection and fault-tolerance techniques.

4.1 CPU and Associated Resources Virtualization

The latest computing devices have introduced *hardware assisted virtualization* solutions (e.g., Intel VT-X and AMD-V use x86 virtualization ring architecture; ARM supports **Virtualization Extension (VE)** and TrustZone) that meet Poper and Goldberg's requirements and significantly impact the hypervisor design since many of the functions provided by the virtualization layer are directly supported by the hardware, therefore reducing the hypervisor code size and computing overhead, and providing direct support for full virtualization because the hypervisor can run in *hypervisor processor mode* and guest OSs in privileged processor mode [222]. Hypervisors and microkernels can use this hardware virtualization support, and it is not necessary to modify the OS. However, although similar in functionality, the hardware support is different and some virtualization solutions are designed specifically for one of these hardware-assisted virtualization solutions. For instance, LTZVisor and variants are exclusively designed for ARM TrustZone [174].

On the other hand, some computing devices (e.g., low-end) lack native hardware virtualization support and the development of software virtualization-based solutions might become a technical challenge [48, 183, 186, 187, 211]. But, most common modern computing devices provide support for at least privileged and user mode instructions and include an MMU [169, 171].

4.2 Peripheral Virtualization

Some computing devices provide native hardware virtualization support for peripherals. For instance, Intel VT-d, AMD-Vi and SPARC V8/V9 architectures provide a **I/O Memory Management Unit (IOMMU)** that enables the (exclusive) allocation and direct use of peripherals to software partitions. While an MMU maps the CPU-visible virtual address to a physical address, the IOMMU maps the peripheral-visible address to a physical address. Alternative approaches have been proposed to support peripheral virtualization in the absence of hardware virtualization or as complementary solutions. These solutions cover I/O virtualization [41, 99, 149, 153], I/O emulation [183] and software virtualization techniques deployed either integrated into the virtualization solution [182, 187], or as specialized partitions. For instance, Reindhart et. al [187] propose the integration of **Virtual Device Emulators (VDEs)** with a hypervisor to support, manage and arbitrate the shared access to peripherals. Other approaches consider the development of hardware-specific I/O server partitions [41, 57] (a.k.a., I/O VM) that exclusively manage a given shared resource and use **Inter-Partition Communication (IPC)** services to communicate with partitions that require access to the given peripheral. Nonetheless, ensuring the required temporal and spatial isolation remains a research challenge, as the usage of shared hardware resources between the CPUs and the peripherals can lead to temporal/spatial interference [34, 38, 49, 149, 153, 169, 171, 222].

4.3 Hardware Observability and Resources Virtualization

In order to support *temporal observability*, the latest HPC devices provide **Performance Monitoring Counters (PMCs)**, **Performance Monitoring Units (PMUs)** and architecture-specific specialized components (e.g., **Event Processing Unit (EPU)** for NXP QorIQ [159], **MultiCore**

Debug Solution (MCDS) for TriCore [209]) that keep track of low-level hardware/software events with high accuracy and low overhead [142, 208]. In this respect, while temporal observability is not a specific requirement in virtualization-based systems, the virtualization layer is responsible for exploiting the hardware-level observability features and making them accessible to the guest OSs and, possibly, to the software application developer [95, 229].

The configuration and exploitation of PMUs and more advanced support, which is relatively straightforward on top of bare-metal setups, gets more complicated in the presence of an OS and possibly multiple ones, in the case of a hypervisor-based setup. In fact, as PMUs are shared hardware modules, it is neither possible to allow multiple configurations to be active at the same time, nor to reserve a quota of the provided support for specific OS images or run-time entities. In this case, it is the responsibility of the hypervisor to *virtualize* the PMU by regulating its configurability and by making its behavior transparent to the virtual environments [65, 130]. Further, different methods to exploit PMU information are possible: counting, sampling, **Processor Event-Based Sampling (PEBS)** and **Last Branch Record (LBR)**, sticking to Intel's jargon. Each method gathers PMU information at different granularity and the method of choice depends on the specific objectives in terms of accuracy and overhead [77, 156]. Additionally, specific hardware support for PMU virtualization is provided as part of wider virtualization support in mainstream COTS platforms, e.g., Hyper-V and VTune integration [95], which are increasingly considered for dependable embedded systems.

For instance, LynxSecure Separation Kernel Hypervisor [217] supports fully virtualized PMCs on targets in the NXP QorIQ family, offering a low-overhead virtualization layer for the on-core PMU. It is not possible to devise whether the supported solution is also covering the EPU module. The solution is already available on the baseline guest scheduler, aka Z-scheduler, and is apparently implemented on top of an interrupt-based sampling mechanism. Additionally, several methods have been proposed for PMU virtualization on top of Xen [238] as summarized in a comprehensive overview [155]. In general, these solutions are more focused on supporting PMU functionalities across virtual environments and do not specifically address the integration with the guest OS. The latter aspect is relevant when it comes to attaching PMU events to specific run time entities (i.e., tasks), which is often the granularity level where observability is required [77].

4.4 Services for Partitions

The virtualization solution must provide services to partitions (e.g., hypercalls) such as interrupt management, **Health Monitor (HM)** and Inter-Partition Communication (IPC) management [85]. Interrupt virtualization and management is key to supporting temporal independence at partition and system level [57, 85] as further described in Section 5.4. And, the HM management is key to supporting reliable and safety systems as further described in Sections 6.1 and 6.4.

IPC services allow partitions to communicate with each other, using basic communication services such as *port-based communication* and *shared memory communication*. In a *port-based communication*, the virtualization solution implements the link between ports by means of channels between partitions. Channels can store multiple messages (queueing channels) or allow a single message at a time (sampling channels). In *shared memory communication*, shared memory areas can be used by partitions to exchange information, and partitions themselves have to deal with the appropriate mechanisms to guarantee the correct access to shared memory information. Overall, virtualization solutions provide heterogeneous and ad hoc IPC implementations [162]. For instance, Xen uses a transport systems mechanism used for device virtualization (front-end and back-end drivers), Jailhouse uses message interrupts in the virtual peripheral component interconnect and XtratuM provides sampling and queuing communication ports [85, 162]. That is why, Oliveira et al. [162] proposed a standardized IPC for TrustZone-based hypervisors. Finally, software developers can also integrate higher-level communication protocols such as **Data Distribution Service (DDS)** [180],

Time-Sensitive Networking (TSN) [38] and **Time-Triggered Ethernet (TTE)** [85] to enable communication among local or remote (deployed in different HPC devices) software partitions.

5 Real-Time Challenges

Real-time requirements are common in dependable embedded systems, with heterogeneity of criticality guarantees such as hard, firm and soft real-time requirements. Hard real-time requirements, such as those of safety-critical functions, represent the highest criticality. The adoption of a specific virtualization solution inherently affects the timing dimension of dependable applications in several aspects. On the one hand, virtualization solutions in the critical domains are typically aiming at enforcing variable (high) degrees of segregation: in multi-core systems, which will typically favor analysability and predictability by reducing the amount of multi-core timing interference. On the other hand, however, the virtualization layer also has implications on how timing information can be extracted from the platform and how timing analysis can be performed (see hardware observability in Section 4.3).

The *partition scheduling* (Section 5.1) offers execution time-slots to partitions according to a partition scheduling policy (e.g., *static*, *dynamic*, combinations thereof). In addition to this, a partition internally schedules internal tasks/threads under the guest OS policy. The partition scheduling configuration and allocation of execution time budgets to critical partitions are commonly based on the partitions WCET analysis (Section 5.2) for a given system architecture, partition scheduling and HPC device. For that purpose, the system architect and software developer must analyze and manage the temporal interference sources that could affect the partitions WCET. For instance, the multi-core device (Section 5.3), interrupts (Section 5.4) and virtualization overheads (Section 5.5). Finally, the integration of temporal diagnosis techniques is a common approach to detect timing interference/degradation before becoming dangerous (e.g., observability of timing-related aspects using performance counters, trapping unexpected interrupts, time constraint monitoring), or just for the detection of such timing misbehavior such as constraint violations or deadline misses (e.g., watchdog) [171].

5.1 Partition Scheduling

Table 2 provides a summary of *static* and *dynamic* partition scheduling approaches supported by listed virtualization solutions. Static approaches are generally preferred over dynamic approaches across leading commercial solutions. Nonetheless, a closer analysis reveals rich and differentiated approaches among virtualization solutions, at least from a research perspective. For example, Gu et al. [81] conducted a survey on real-time schedulers in Xen, KVM and L4-based micro-kernels.

Static table-driven approaches are robust and easy-to-validate approaches recommended by functional safety standards (e.g., IEC 61508-3 Annex F). Thus, this is a common approach for virtualization solutions that target the development of safety-critical systems (e.g., μ -visor, Multivisor, PikeOS, XtratuM/XNG). The scheduling table is defined at design time. And, it is used at run time to regulate partition execution time using precomputed execution windows or slots. Although static table-driven schedulers offer the highest degree of predictability, they often lack flexibility and a single scheduling change request might require a complete table redesign [85].

On the other hand, *dynamic* approaches can also provide predictability and analysability properties with a more flexible scheduling scheme executed during run-time. For instance, the *dynamic fixed priority preemptive* scheduling scheme can support complex task models, taking into consideration detailed timing characteristics (e.g., overheads) in the schedulability analysis. These approaches still rely on static schedulability analysis techniques to assess whether a system is schedulable but they differentiate from table-driven approaches in that no explicit schedule is constructed. At run-time, partitions are executed according to their associated priority. Other solutions are built on

dynamic planning-based approaches to verify scheduling feasibility at run-time, which means that a partition is only accepted for execution if it is deemed feasible. Finally, *server schemes* combine or centralize the workloads of various physical execution environments in a computing device using a server abstraction to limit the amount of CPU dedicated to each partition [85].

Overall, virtualization solutions support an evolving heterogeneity of static and dynamic schedulers, including combinations thereof. For instance:

- μ RTZVisor implements a server-based scheduling approach in which partitions are assigned a constant bandwidth. The scheduling plan is organized into domains with a constant bandwidth scheduled under Round-Robin algorithm (where domain 0 has the highest priority) [136, 173].
- Xen provides three CPU schedulers: BVT (Borrowed Virtual Time, now deprecated), SEDF (Simple Earliest Deadline First), and Credit which is the default scheduler. The Credit scheduler uses a proportional-share algorithm to balance the workload of vCPUs across physical cores in a multicore processor. Additionally, it offers automatic workload balancing. On the other hand, SEDF is a server-based scheduler that allows each VM to reserve a portion of the CPU bandwidth, and it also features a boolean flag that indicates if the VM can receive extra CPU time. And, the RT-Xen project [240, 241] has developed a hierarchical real-time scheduling framework for Xen. Finally, since 2017, it is also possible to use the ‘null’ scheduler in which each vCPU is statically assigned a physical CPU. If there are more vCPUs than physical CPUs, they do not execute until they are assigned.
- KVM’s default scheduler is Round-Robin. Being a Type 2 hypervisor somehow limits the real-time improvements to those made on the Linux host that must use, at least, the PRE-EMPT_RT patch. The IRMOS scheduler [59] implements a server-based approach, specifically the **Constant Bandwidth Server (CBS)** algorithm, on top of **Earliest Deadline First (EDF)** scheduler in Linux. Finally, the RESCH scheduler ported to KVM [247] supports four fixed priority-based algorithms.
- PikeOS uses a combination of priority and time-driven scheduling and proportional share scheduling [107]. This combination ensures that hard real-time requirements are met (critical tasks) while providing best-effort scheduling for other tasks (non-critical). For instance, the time-driven scheduler distributes the available CPU time among the partitions.

5.2 Worst Case Execution Time (WCET) Analysis

As previously explained, the correct scheduling configuration and correct allocation of time budgets to partitions require a careful analysis (e.g., schedulability analysis) that is commonly based on a WCET analysis. However, as summarized by Abella et al. [11] and embedded HPC device surveys [169, 171], the WCET analysis of software applications deployed on COTS HPC devices designed for maximum average performance is a technical challenge. Especially for safety-critical systems that require safe estimates. Multiple sources of timing interference can potentially affect the execution time of partitions and therefore must be taken into account for the determination of trustworthy timing bounds, such as the HPC device itself (Section 5.3), interrupts (Section 5.4) and computing overheads of the virtualization solution (Section 5.5). Nonetheless, several techniques have been proposed for the WCET of partitions such as static and measurement-based analysis methods (e.g., **Probabilistic Timing Analysis (PTA)** [11, 13, 120]). And WCET hypotheses/estimations can be tested at a later stage with the definition of measurement-based validation tests [120].

5.3 HPC Devices Temporal Interference Management

The problem of timing interference in embedded HPC devices (e.g., MPSoCs, GPUs) is in many ways relevant for hypervisor-based systems [169, 171]. On the one hand, the temporal and spatial

segregation solutions, typically enforced in hypervisor-based high-integrity systems, are certainly constraining the amount of timing interference potentially arising during execution. Not surprisingly, hypervisors are widely considered as the reference solution for deploying high-integrity systems on MPSoCs [66, 164, 228, 235] and GPUs [43]. On the other hand, hypervisor-based approaches build on the hypervisor's capability to offer a clean separation of execution contexts for the different software partitions. Such separation in MPSoCs with abundant (stateful) resource sharing requires a combination of hardware and software support [3, 19, 64, 66, 145, 228, 239] in order to preserve the performance improvements such platforms are meant to provide. Software and hardware level solutions are increasingly proposed by chip designers and hypervisor/OS providers for controlling how complex shared hardware resources are shared in space and time, with a view to avoiding or greatly reducing the impact of interference. For instance, even hardware/software-assisted peripheral virtualization solutions (e.g., I/O virtualization) described in Section 4.2 require a careful temporal interference analysis [49, 116].

Hardware-level support for resource partitioning is the preferred solution to guarantee isolation across software partitions without the need of resetting the resources' hardware state upon partition switches, hence incurring inherent performance degradation. For example, the NXP T2080 MP-SoC [160] provides highly-flexible hardware level control for the operation of shared L2 cache. The **Memory System Resource Partitioning and Monitoring (MPAM)** [19] extension supporting cache partitioning has been introduced in ARM-V8 architectures (precisely from V8.4). Support for governing the shared resources' utilization is provided by Intel **Resource Director Technology (RDT)** [94]. When available, hardware-level partitioning support can be straightforwardly managed and exploited by the hypervisor layers to effectively implement isolation and avoid or reduce multi-core timing interference, e.g., LynxSecure with T2080 L2 partitioning [135].

However, not all targets support the partitioning of shared resources and, even when present, partitioning support may not be provided at the correct granularity. As an example, the NXP Layerscape LX2160A [161] does not support partitioning the L2 cache, which is shared at the CPU cluster level. In the lack of hardware support, partitioning and interference mitigation can be implemented at the hypervisor level by software. Cache coloring techniques can be efficiently implemented at the hypervisor level [115, 239] on top of existing address translation schemes and with specific hardware support to **second-level address translation (SLAT)**. An assessment of different coloring solutions and the entailed overhead, from the perspective of real-time performance guarantees is provided by Kloda et al. [115].

Based on the publicly available information, it is reasonable to assume virtual cache coloring solutions are also apparently at the basis of commercial hypervisor-based solutions. For instance, safety-critical RTOSs use equivalent techniques such as memory pools in DDC-I Deos [64] and **Bandwidth and Allocation Manager (BAM)** [3] in GreenHills Integrity. The latter, in particular, is also exploiting bandwidth management to monitor and regulate the use of shared resources by controlling and possibly limiting, for example, memory accesses from a particular core. Bandwidth management for controlling multicore interference, originally introduced in [157, 158] on top of a PikeOS integration on NXP T2080, builds on the capability of collecting hardware events related to memory resource usage. These approaches are most effective in a combination of partitioning/segregation support and deep integration between the hypervisor and the guest OSs as bandwidth regulation typically happens at task granularity (hence at the OS level) while isolation is better typically enforced at the granularity of software partitions, hence at the hypervisor level.

When it comes to more complex resources, such as interconnects and accelerators, resource partitioning is not always possible and, equally relevant, convenient in terms of performance guarantees. Chip vendors are providing increased support for **Quality of Service (QoS)** setups, especially addressing memory controllers and interconnects (e.g., [17, 18]). Advanced QoS features

can be exploited not only to improve average performances but, when properly controlled, also offer opportunities to provide execution guarantees [198, 246]. QoS control is typically offered through static and dynamic mechanisms. Both of them can be properly managed at the hypervisor level. A proof of concept QoS management on top of Jailhouse hypervisor is presented in [246].

5.4 Interrupts Temporal Interference Management

Inappropriate interrupt handling can generate non-negligible timing interference with the effect of jeopardizing the required execution determinism of software partitions (i.e., by making execution dependent on the sequence of interrupts and handlers) or, causing software partitions to exceed their assigned time budget [223]. One conservative approach to reduce the impact of interrupts across partitions consists in making each partition periodically poll for its own exclusive interrupts or reserving a dedicated software partition that manages shared interrupt sources and forwards them via IPC to interested software partitions [57, 148, 223]. However, these conservative approaches might be resource-consuming and inefficient compared with other interrupt-handling approaches [148]. For instance, “taming” is a dynamic interrupt handling approach with a dynamically decreasing time budget [223] and Safeguarding Interrupts (SgInt) is a low-overhead safe interrupt handling approach that manages the I/O devices (e.g., PCIe) internal interrupts (e.g., message-signaled interrupts) in computing devices without IOMMU support (or equivalent) [148].

In general, low-latency interrupt handling is a requirement in dependable systems, and virtualization solutions support different safe approaches to minimize interrupt impact that are tailored to the internal architecture or expected usage. For example, seL4 supports preemptive interrupt handling at explicit software execution points [40] to provide a reasonable interrupt latency while supporting the formal verification and WCET analysis of the kernel; Xen provides a special domain for interrupt handling prior to the redirection to the guest [233]; Quest-V manages interrupts directly in each sandbox rather than monitors to reduce latency [233] and a modified MicroC/OS-MMU handles interrupts in other software partitions with an interrupt-monitoring solution [29].

5.5 Analysis and Characterization of Virtualization Overheads on Execution Time

All hypervisor-based solutions build on the introduction of a more or less rich and intrusive layer in order to provide the desired degree of segregation and predictability. While being fundamental for delivering the hypervisor functionalities, this layer is also undeniably affecting the overall system performance. Several studies have been attempting to compare and classify hypervisor solutions based on the performance overhead they incur. This is especially true in mainstream domains where performance is one of the most prominent factors when selecting among different hypervisor solutions. For this reason, the large majority of comparative assessments fall in the scope of cloud services and data-center solutions [146, 207, 230] and consider hypervisor solutions that are not typically deployed for dependable embedded systems.

Performance is a major concern also in dependable embedded systems in that it relates to response time guarantees. Several works have been focusing on the overhead incurred by specific hypervisor implementations. Most of these studies contribute an empirical characterization of the overhead induced by a specific hypervisor implementation [33, 56, 115, 152, 187, 233, 244]. Relatively fewer studies attempt to derive precise response time bounds, especially focusing on I/O virtualization [50, 116] under the assumption that the segregation guaranteed by the hypervisor setup is not hindering timing analyses.

The performance overhead incurred by XtratuM on a multicore mixed-criticality scenario is addressed in [56]: the focus of the study is mainly set on the impact of XtratuM management of software partitions, especially when multiple cores are exploited, thus possibly causing timing interference. At the partition level, the performance degradation is extremely low in absolute

terms and generally below 1% even for short partitions. Results are also reported on a synthetic exploration of multicore interference on the specific platform, suggesting that the underlying static schedule can be exploited to mitigate the impact of contender partitions by controlling how much those partitions are allowed to overlap in time. The performance overheads incurred by XtratuM for space on an RTEMS SMP setup are addressed in Ref. [112] on LEON3 and LEON4 targets, reporting negligible overheads in the general case, with few exceptions on very specific benchmarks.

Still in the scope of multicore systems in the space domain, the authors in [152] look at hypervisor-based solutions as the most natural way to support segregation requirements but at the cost of performance degradation. A systematic approach is presented for assessing the performance of different hypervisors together with other metrics such as isolation and footprint. The approach is deployed to compare PikeOS and XtratuM on top of LEON3 and LEON4 platforms, where the two solutions are reported to exhibit similar overheads.

Both empirical and analytical studies have addressed the analysis of different hardware-supported I/O virtualization approaches supporting memory segregation over the I/O subsystem. Hypervisor solutions relying on IOMMU mechanisms are addressed in [33] where the overhead imposed by such type of MMUs for disk and network I/O operations is assessed on the Xen hypervisor. The authors show that enabling IOMMU on Xen is only negligibly affecting the system throughput but implies a sensible increase in CPU utilization (up to 60% compared with the IOMMU disabled scenario). The overhead, however, can be effectively reduced via software optimization.

Analytical bounds for different I/O virtualization techniques (pass-through and paravirtualization) are provided in Ref. [50]. The proposed model is flexible and adaptable to different hardware and software setups and provides a detailed insight into the different sources of overhead, paving the road for further optimizations. A similar effort has been reported in [116] for the timing characterization of hypervisor-based I/O virtualization. In this case, observed overheads are compared against a legacy AUTOSAR implementation showing how overheads are strongly related to the actual I/O virtualization approach and scheduler configuration. All these approaches share a common conclusion that overhead is extremely dependent on configuration and therefore adaptations to match application requirements and semantics are often necessary.

A performance comparison between Xen and Jailhouse hypervisors is presented in [244] motivated by the increasing penetration of such solutions in the embedded systems domain. The two solutions are evaluated against a baseline Linux version on top of a system equipping ARM Cortex-A7 CPUs. Results show the Xen hypervisor is introducing non-negligible overhead compared with baseline Linux (especially when a credit scheduler is used) whereas the overhead incurred by Jailhouse is extremely low, motivated by the lack of a real scheduler implementation. Performances however come at the cost of a reduced set of services compared with Xen. It is worth noting that similar results were also reported in an older study [220], still building on ARM support and providing similar conclusions, especially highlighting the negligible overhead incurred by Jailhouse. An empirical assessment of latencies in the Quest-V separation kernel against those exhibited in the Xen hypervisor is provided in [233] showing how hardware virtualization support can be exploited without incurring the overhead of traditional hypervisor solutions. A comparative assessment of the overhead introduced by KVM and Xen is assessed in [12] with a view to making hypervisor latencies compatible with real-time requirements.

In the automotive domain, microkernel-based virtualization is considered in [187] as a means to complement a (paravirtualized) AUTOSAR basic software with better support for safety and failure propagation avoidance in mixed-criticality automotive setups, without resorting to MMU support. However, despite the benefits in terms of segregation, integrating hypervisor support on top of the support available in an AURIX platform and AUTOSAR setup is pretty onerous and the performance degradation incurred does not seem to always justify the effort.

A family of works is looking into custom hardware or software solutions to favor the predictability and analysability of virtualization solutions, again with the main focus on I/O management [100, 101, 141, 147, 149, 153]. The implementation of a custom hardware component to enable direct I/O requests bypassing the guest OS, known as **Virtualized Complicated Device Controller (VCDC)**, on BlueVisor is described in [100]. Software solutions for improved predictability of virtual I/O management for critical real-time systems in the lack of IOMMU support are discussed in [101, 149, 153]. Another approach combining memory-mapped I/O, MMU and IOMMU that improves analysability and real-time performance is presented in [147]. In Ref. [141] improvements over the standard Xen scheduler are proposed to favor predictability, together with the concept of real-time VMs, as a means to reduce latency and jitter for real-time applications. A similar approach, extending Xen scheduler capabilities to favor predictability is further developed in [97].

6 Dependability Challenges

As outlined in Section 1, dependability attributes are common requirements in industrial and transportation systems and additional domains outside the scope of the survey (e.g., mobile, cloud computing) that also use virtualization technology, resulting in a large number of research contributions. This Section summarizes the selected contributions to dependability aspects of virtualization technology and the use of this technology in the industrial and transportation domains.

6.1 Reliability

Virtualization solutions execute with the highest privileged modes of the processor device and complete hardware access [114, 244]. Thus, design-time systematic errors and execution-time random errors can lead to an incorrect operation of the virtualization solution and the overall system built on top of it. Therefore, virtualization solutions tend to be small with minimum services, to reduce the amount of privileged software code that could lead to systematic or random errors [57, 114] with sizes that can vary from approximately ten thousand to a few hundred thousand LoC (see Table 2). For example, in the avionics domain, experts agree that for high robustness evaluation, “a component must be small, on the order of 5,000 lines of code” [113].

6.1.1 Systematic Errors. Virtualization solutions are commonly developed using or extending common techniques for reliable software development. For instance, using coding standards (e.g., Xen: MISRA-C), performing reviews/analyses (seL4: IPC analysis of architectural properties and dependencies [125]) and testing-based verification campaigns (e.g., XtratuM [57], seL4 [214]).

Besides, *formal verification* can provide a “complete” correctness verification coverage of both the design and the software implementation, and has attracted significant research activity as summarized in multiple surveys/reviews [24, 35, 245]. The prominent example is the seL4 kernel, which has been formally verified at the abstract design level (Haskell language) and software implementation levels (C language), ensuring that the kernel is free of design and programming errors under a certain set of reasonable assumptions [90, 114]. Additionally, multiple virtualization solutions have also been formally verified for reliability, safety, or security compliance purposes (e.g., PikeOS [26, 27], SafeBox [243], seL4-based design initialization [42]), using formal methods and languages such as Isabelle/HOL [114, 243] and VCC [14, 26, 27].

However, the effort required to perform a formal verification is significant (e.g., seL4: 25 person-year for functional correctness) [114], and it is only considered technically and economically feasible for small-footprint solutions (e.g., less than 10kLoC [224]). Therefore, formal verification has only been extensively used with seL4 and only partially deployed for several other solutions. As detailed in several studies, KVM for ARM has undergone partial formal verification. These include formally verifying the correct handling of ARM processors’ relaxed memory model [213], memory

protection and isolation across different virtual machines [126], and critical security properties to prevent vulnerabilities and attacks [127]. In addition to this, formal verification of properties has also been applied to simplified models of COTS hardware-assisted virtualization solutions such as Intel VT-x [34].

6.1.2 Random Errors. Dependable embedded systems commonly integrate *diagnostic* and *fault-tolerance* techniques to detect and control random hardware errors. Concerning *diagnostics*, virtualization solutions for dependable embedded systems commonly integrate a Health Monitor (HM) module that centralizes the detection of anomalous events, states, or random hardware errors (e.g., exception management, unused interrupt trap, memory corruption). The HM executes a design-time predefined error reaction (e.g., partition execution halt) [57, 85, 167, 168, 222]. And the HM builds on top of the hardware built-in techniques provided by each HPC device (e.g., memory ECC) [169, 171]. On the other hand, *fault-tolerance* techniques commonly rely on redundancy [69, 85, 144, 167, 168, 200] and diversity [85, 167, 168] of computing channels, software partitions and virtualization solutions. And the effectiveness of both the implemented diagnostics and fault-tolerance techniques can be evaluated through fault-injection campaigns [31, 54, 69, 71, 72, 193].

6.2 Maintainability

Maintainability attribute is commonly described as a potential advantage of virtualization-based systems [92, 185, 187]. Still, few research contributions provide a relevant technical contribution to system maintenance improvement using virtualization technology [185]. For example, Reeser et al. [185] explain the application of virtualization technology for maintaining SCADA and **Human-Machine Interface (HMI)** industrial systems. And Holstein et al. [92] explain the advantages (e.g., maintainability) and disadvantages, of using hypervisor solutions for the development of vehicle infotainment systems that combine functions of different safety criticality.

6.3 Availability

Availability attribute is also commonly stated as a potential advantage of virtualization-based systems [24, 25, 57, 102, 103, 132, 134, 172, 185, 192, 195] by means of, for example, run-time software partition migration support. In high-availability systems, software partition migration enables run-time hardware upgrades [192], backup and recovery [132], and dynamic system reconfiguration [97]. Furthermore, run-time safety software partition migration capability in safety-critical systems provides a fault-tolerant technique to increase the overall system availability [134].

6.4 Safety

The development of safety-critical systems relies on stringent methods and conservative assumptions that can lead to underutilization of hardware and system resources [167, 222]. For instance, when the provided temporal and spatial independence is inadequate, or as required by certain standards (e.g., ARINC 653), the execution of safety partitions may limit the software execution to a single core in a multi-core device [171]. Therefore, developing virtualization solutions that provide temporal and spatial isolation in compliance with safety standards, with a low-performance overhead, and maximizing resource utilization is a technical and research challenge. On top of that, in some specific applications, there might be additional non-functional requirements that condition virtualization solutions and their use for the development of safety-critical systems. For instance, some specific transportation and industrial safety-critical systems must also operate with low-power constraints, requiring specialized solutions and techniques. For example, remote railway interlocking object controllers must support the highest level of safety while operating with

low power constraints ensuring that implemented low-power techniques (e.g., power-off certain hardware resources) do not jeopardize functional safety requirements [73, 177].

In the remainder of this Section, we follow the argument structure of a modular safety case for a generic IEC 61508 compliant hypervisor [123], integrating the argumentation used in diverse safety concepts [13, 167, 168, 181] and safety patterns [121, 122]. Moreover, the most common focus of research contributions is safety assurance but not safety certification in strict compliance with safety standards, which is commonly considered an industrial challenge. Furthermore, the safety-critical system designer develops the system based on a set of constraints, fault hypotheses and assumptions that must be explicitly stated [13, 167, 168, 181]. For instance, “the hypervisor is a compliant item, provides interference freeness among partitions bounded time and spatial isolation), supports temporal isolation techniques [93] and fails in an arbitrary failure mode” [168], and “the system configuration is static and defined during the design time” [222].

6.4.1 Safety Standard Compliance. As a rule of thumb, the usage of certified/qualified virtualization solutions simplifies the development and certification of safety-critical systems. However, as summarized in Table 2 and described in detail in a specialized survey [131], only a few virtualization solutions have been developed and certified/qualified in compliance with applicable generic and domain-specific safety standards. And, additional ones are reported to be under certification processes (e.g., L4Re [118]) or researching in this direction (e.g., Jailhouse [53]). In all cases, the virtualization solutions tend to provide the minimum set of services to reduce the software code size subject to safety certification [224].

Concerning the safety-critical system development and certification, Table 3 (page 25) provides a summary of example safety-concepts [13, 167, 168, 181], safety reference architectures [38, 69, 80, 82, 91, 110, 144, 224, 227], use cases (e.g., automotive [36], avionics [184]) and safety patterns [84, 121, 122, 184, 242] that contribute towards the development of virtualization-based safety-critical systems in compliance with applicable safety standards.

Formal verification, already explained in the reliability section (Section 6.1), is a highly recommended technique for the development of safety-critical systems with the highest criticality levels (e.g., IEC 61508-3 Table A.5 [93], DO-178 [26, 226]). Nonetheless, just the strong mathematical foundation used for the formal verification might not be sufficient for safety standard compliance [226]. Even hypervisors already formally verified might have some limitations whenever extending the formal verification to the safety-critical domain. For example, tool qualification requirements [93, 226], the detailed platform-specific formal verification must be revisited when using different computing devices (e.g., seL4 for ARMv6 i.MX31 [114, 226], PikeOS for PowerPC MPC52200 [26]), ensuring imposed real-time performance constraints (e.g., WCET, interrupt handling latency [40]), the specific C/ASM software implementation should also be verified [196] and the detailed functional integration with underlying hardware components must also be verified (microkernel and MMU based spatial isolation [106]). On top of that, all formal verification assumptions must be verified, and some assumptions might not be possible to hold with modern computing devices (e.g., seL4: excluded the usage of DMAs [226]).

Finally, safety engineers can benefit from available tools for developing virtualization-based safety-critical systems. For instance, several research contributions contribute with **Model Driven Engineering (MDE)** tool-chains [85, 138, 151, 222] that support diverse development activities such as partition schedulability analysis [85], verification of design properties such as real-time constraints [57, 85, 222] and system configuration (e.g., partitions, resources, HM). Nonetheless, tool qualification requirements shall be evaluated.

6.4.2 Resource Virtualization and Exclusive Access to Peripherals. As a general rule, the design-time, static and exclusive allocation of resources is a recommended technique for the development

of safety-critical systems [57, 93, 110]. And, during execution, the virtualization solution shall guarantee that a software partition can access the assigned resources and detect and deny access to other resources [167, 168, 222]. Example resources are vCPUs, memory regions and peripherals (see Section 4). However, as explained in Section 4, in some cases, a resource cannot be exclusively assigned to a software partition. For example, shared peripherals (e.g., communication protocol) and peripherals accessed in a shared memory region (e.g., I/Os registers) in the absence of hardware-assisted virtualization support such as IOMMU or TrustZone. In this scenario, a possible solution is the temporal and/or spatial domain partitioning of shared resources [224], so that software partitions can exclusively access them at given time slots of spatial slots (e.g., memory region). Another possible solution is the development of server software partitions that safely manage these shared resources and communicate with client software partitions to exchange commands and information (e.g., I/O server [111]). For example, Jailhouse supports sharing memory pages, but with no protection [184], and the protection must be implemented by the system architect implementing, for example, previously described spatial division managed by developer-defined software partitions or client-server approaches.

6.4.3 Safe Startup and Safe Shutdown. The virtualization-based safety-critical system and the virtualization solution itself shall startup in a known, bounded time and repeatable logic sequence [110, 123, 168, 184, 211]. During the safe startup, the processing device and the virtualization solution shall be configured and initialized before a guest virtualized environment starts execution and non-critical partition(s) should not interfere with the safe startup of other partitions [168]. For example, the safe startup process has also been formally verified for a seL4-based design [42]. Moreover, the safe shutdown shall also ensure a safe shutdown of critical partitions, virtualization solution and underlying processing device to ensure that safe states are correctly applied (e.g., de-energization of digital outputs, fail silent communication) [123].

6.4.4 Temporal Independence. The virtualization solution shall provide sufficient processing time for safety partitions to execute on given deadlines, ensuring that partition time-budget and schedule are assigned as defined during design-time (e.g., static configuration) [167, 168]. For that purpose, the system architect must assign a sufficient partition time budget, perform WCET analysis, analyze and mitigate temporal interferences, select and configure the appropriate partition scheduling and deploy temporal independence diagnosis to confirm that required temporal guarantees are met.

The execution of one software partition shall not affect the performance of other unrelated software partitions [224]. However, as described in the real-time section (Section 5.3), the execution of software in HPC computing devices with shared resources leads to several challenges regarding timing interference [116, 137, 181, 218]. And partitions executed on multi-core devices could be subject to temporal interference that could lead to potentially dangerous risks such as missing temporal deadlines. Nonetheless, different techniques can avoid, reduce or mitigate these temporal interference sources such as memory bandwidth reservation [137, 169, 171], cache partitioning (e.g., coloring), I/O virtualization with time predictability [38, 41], analysis of scheduling plans and potential communication performance impact [218] and interrupt management [38].

As summarized in Table 2, virtualization solutions support diverse partition scheduling algorithms. And virtualization solutions targeting safety-critical systems commonly support design time-defined static cyclic schedulers [57, 167] in compliance with safety standards that request or recommend their usage (e.g., IEC 61508-3 Annex F). Furthermore, in order to guarantee temporal deadlines and simplify temporal analyses, virtualization solutions tend to be non-preemptive [57, 167] or allow preemption at explicit execution points (e.g., interrupt handling) [40].

Finally, temporal independence constraints must be periodically confirmed to ensure that temporal guarantees are met by means of temporal independence diagnosis (e.g., temporal and logical program sequence monitoring for safety tasks/partition [122, 168, 181]).

6.4.5 Spatial Independence. To ensure that software partitions do not interfere with one another, virtualization solutions must also provide spatial independence by preventing one partition from overwriting another's data or accessing memory not explicitly assigned to it [93, 168]. For that purpose, software partitions are commonly allocated exclusive access memory areas during the design phase [57, 222], and system architects can also define shared memory areas between partitions for communication purposes [222]. Additionally, virtualization solutions are commonly executed in the device privileged/supervisor mode and user partitions in the user mode [57]. The virtualization solution relies on underlying specialized hardware components such as MMU and **Memory Protection Unit (MPU)** [87, 167, 168, 181, 187, 211, 233], or security-specialized components (e.g., TrustZone), provided by computing devices to ensure spatial isolation among software partitions and detect violations [169, 171]. Nonetheless, periodic diagnosis is also necessary to confirm that spatial independence holds during runtime. For example, checking that the hypervisor code has not been modified, the checksum of protected memory blocks [122] and developing application and device-specific diagnosis techniques (e.g., cache coherency management diagnosis [122]).

6.4.6 Safe Inter-Partition Communication (IPC). To support the safe communication among safety partitions, the IPC (Section 4.4) shall, for instance, use a safe communication protocol (e.g., TTE [85]), a **Safe Communication Layer (SCL)** [122, 236] or implement a safe shared memory-based protocol. In addition, the IPC might also implement additional diagnostics, flow control and fail-safe properties to ensure that erroneous software partitions (e.g., "babbling idiot") do not jeopardize the communication bandwidth and temporal deadlines required by other safety partitions [85, 110].

6.4.7 Diagnosis and System Reaction to Errors. Functional safety standards have stringent error detection requirements (diagnosis) and diagnostic coverage threshold requirements [171]. Thus, this leads to stringent requirements for the HM module described in subsection 6.1.2 and the usage of dedicated partition(s) [222] and core(s) [134] for such purpose. For example, VOSYSmonitor uses a dedicated core for self-tests and diagnosis [134]. Furthermore, the system architect must define the system-level diagnosis and system reaction to errors based on the services and built-in mechanisms already provided by the virtualization solution and the underlying computing device (e.g., multi-core device [171], GPU [169]). Taking into consideration that software partitions might be provided by different suppliers, it is key for the system architect to define an overall strategy to reduce the emerging complexity of an integration of multiple software partitions in a complex HPC device [181, 222]. For instance, Perez et al. [167, 168, 181] recommend system architects define a system-level strategy for the deployment of diagnosis techniques, where partitions should be developed abstracted from the underlying host, and specialized host-specific partitions should implement required complementary diagnosis not provided by the virtualization HM.

6.5 Integrity

The virtualization solution itself should be isolated and protected from other software partitions (and OSs). For that purpose, the hypervisor executes in the highest privileged mode [167, 168]. Additionally, a high diagnostic coverage should be provided to detect execution random errors integrating techniques previously described (e.g., checking with a checksum that the hypervisor code has not been modified [134]). And, the virtualization solution shall ensure that errors in one software partition are isolated from other partitions [224].

In recent years, the increasing connectivity of systems, particularly through the Industrial Internet of Things (IIoT), has increased the potential attack surface for malicious actors, making cybersecurity a critical integrity concern for developing dependable embedded systems for transportation and industrial domains. For example, the latest safety standard updates/drafts explicitly

require security measures for developing safety-critical systems (“no safety without security”) [242]. This is also reflected in the thousands of research contributions focused on virtualization and security² and security-specialized virtualizations solutions (e.g., Hafnium [119]). The convergence of dependability and security requirements leads to specialized architectures such as the Multiple Independent Levels of Security / Safety (MILS) architecture [113], commonly implemented as a *separation kernel*, specialized techniques/measures for the development of virtualization solutions (e.g., formal verification to prove the absence of security vulnerabilities [150, 226]), and specialized techniques/measures for the operation and maintenance of systems (e.g., remote updates [231, 242]).

7 Application Domains and Case Studies

Virtualization technology offers several generic benefits and advantages of interest for developing dependable embedded solutions. However, as summarized in this Section, the industrial and transportation domains have unique requirements requiring specialized hypervisors, features and usage patterns. Therefore, as summarized in Table 3, multiple virtualization solutions have been evaluated using domain and application-specific research case studies, performance evaluations, reference architectures and safety concepts.

7.1 Transportation Domain

In the following, we focus on hypervisor solutions and case studies in the automotive, railway, avionics and space domains.

7.1.1 Automotive. “Premium cars have more than 20 million lines of code deployed in” over 150 ECUs [171, 211], driving up electronics costs from 20% to 35% [110]. The development of ADAS and autonomous driving systems further exacerbate the need for integrating more software functionalities that require diverse OSs, complex software stacks and higher computational performance devices [169–171, 201]. However, the automotive domain is driven by the economy of scale, and cost-optimization requirements are leading manufacturers and OEMs towards an integrated architectural approach with the definition of computationally powerful **Domain Controller Units (DCUs)** and the integration of multiple ECUs [154, 187]. In this context, AUTOSAR and virtualization solutions enable the integration of multiple mixed-criticality software applications in a single HPC device [187] with multiple potential benefits for the automotive domain such as OS multiplicity, hardware longevity and HPC devices support [210]. Moreover, AUTOSAR flexibility for the integration of software components can be combined with the temporal/spatial isolation foundation provided by virtualization solutions for such integration [9, 86, 87, 118, 183].

Table 3 summarizes selected research contributions that target the automotive domain providing reference architectures [32, 36, 43, 206], case-study examples [39, 110, 154, 178], and evaluation and analysis examples [63, 83, 183, 186, 187, 211]. Additionally, virtualization solutions specialized for the automotive domain commonly support AUTOSAR or comply with the automotive domain ISO 26262 safety standard (see Table 2).

7.1.2 Railway. Virtualization technology and multi-core devices are being increasingly adopted in railway systems for developing safety-critical systems. To evaluate the feasibility of this approach, SIL4 railway signaling case studies have been conducted using solutions such as PikeOS and XtratuM [13, 37]. For instance, Bilbao et al. [37] describe the safety concept of a railway object-controller safety-critical system (SIL4) with safety and low power requirements. In this particular safety concept, they propose the usage of several scheduling plans associated to different energy

²(TITLE-ABS-KEY(virtualization) OR TITLE-ABS-KEY(hypervisor) OR TITLE-ABS-KEY(microkernel)) AND (TITLE-ABS-KEY(security) OR TITLE-ABS-KEY(cybersecurity)) AND (LIMIT-TO (SUBJAREA, “COMP”)).

usage degraded modes managed by a master partition. Finally, PikeOS is the only virtualization solution listed in Table 2 that complies with the railway domain software functional safety standard.

7.1.3 Avionics. In the avionics domain, during the last few decades the **Integrated Modular Avionics (IMA)** architecture has been used by most avionics systems to share hardware and software resources, enabling the integration of more system functions in fewer computers and reducing the overall SWaP of computer systems [171]. Nonetheless, the increasing performance demand, particularly for new ML-based applications such as Urban Air Mobility, Cargo Drone Delivery [70] and **Unmanned Aircraft Systems (UASs)** [194] require multi-core devices to support the required computational performance without increasing the number of on-board computers [171]. This leads to the multi-core integration (multi-IMA) of legacy single-core IMA systems where the temporal and spatial isolation of legacy partitions must be guaranteed, particularly for safety-critical software partitions. The use of virtualization-based safety-critical systems in compliance with applicable safety standards and including safety concepts (e.g., [1, 188, 191]), is mandatory for highly robust integration and resource management in multi-core IMA systems.

Regarding the MILS architecture and avionic security, Kleidermacher et al [113] proposed a MILS-based hypervisor architecture for IMA systems that provides strong isolation between different avionics applications, ensuring that faults or security breaches in one application do not affect others. This study demonstrates that the use of hypervisor technology in IMA systems can improve the security and reliability of avionics systems by preventing unauthorized access to sensitive information, detecting and preventing cyberattacks, and ensuring that the failure of one application does not affect the operation of other applications. This approach has been widely adopted in the aerospace industry, and several studies have demonstrated the benefits of using hypervisor technology in IMA systems. Overall, the application of hypervisor technology to IMA systems can significantly enhance the security and reliability of avionics systems, making them more robust and resilient to cyberattacks and faults.

Table 3 summarizes selected research contributions that target the avionics domain describing IMA architecture and partitioning approaches [75, 184, 226], case-study examples [194], generic aspects and arguments for safety systems [224, 227]. Finally, virtualization solutions specialized for the avionics domain commonly support ARINC 653, support the IMA architecture, or comply with avionics domain safety standards (see Table 2).

7.1.4 Space. In space systems, the reduction of SWaP is also important. Therefore, similar to the avionics domain, the space domain has also developed the IMA for space (IMA-SP) architecture, which consolidates multiple software functions of different criticalities into a reduced number of computers. Additionally, space systems are deployed in harsh space execution environments (e.g., high level of radiation) that commonly require hardened specialized processors (e.g., LEON3), along with fault-tolerance and high diagnostic coverage solutions to enable reliable execution of software applications. The size and complexity of these software applications can scale by a factor of 10x every 10 years [171]. In this context, radiation-hardened multi-core devices and virtualization technology become a foundation for supporting next-generation space systems' development [169, 171]. They enable software application partitioning and time-scheduling of partitions with limited overhead in compliance with space standards (e.g., ARINC 653). For instance, the XtratuM/XNG hypervisor is widely used in space applications (deployed on 76 satellites at the time of [232] but on more than 700 at present), and it is the only virtualization solution listed in Table 2 that complies with space ECSS standard. Additionally, as summarized in Table 3, several research studies have utilized this hypervisor to evaluate fault-tolerance strategies [45], time overhead [112] and comparative benchmark analyses [152].

Table 3. Summary of Domain-Specific Case Studies, Architectures and Analyses

Domain	Arch. / Case Study	Hypervisor	Description
Automotive	Generic	LWHVR	Hypervisor evaluation and analysis using diverse automotive applications (e.g., engine, vehicle control, communication controller) with a hypervisor that does not require virtualization support of the underlying automotive safety device (TC27x, TC29x) [63, 183, 186, 187, 211].
		Jailhouse	Software stack reference architecture for mixed-criticality applications deployed in HPC device(s) [43]
		Multivisor, PikeOS, QNX Hypervisor	Hypervisor selection and evaluation method for automotive applications [83]
		Quest-V	Generic vehicle management system architectural approach based on a specialized Quest-V virtualization solution (DriveOS) with diverse guest operating systems (Linux, Android) [206]
	Cockpit,	QNX Hypervisor	Hypervisor-based digital cockpit supporting the integration of mixed-criticality functions [178]
	Infotainment, HMI	Xen	The automotive cockpit integrates three software partitions in a multi-core device. Two non-safety-related software partitions (with Linux and Android OS) execute with Xen, while the safety partition (with TI-RTOS) does not execute with a hypervisor [110].
		Jailhouse	Automotive software stack example that integrates HMI and real-time tasks using an hypervisor, AUTOSAR and heterogeneous OSs (e.g., Linux, Erika) [39]
	DCU	-	Timing characterization of hypervisor-based I/O virtualization (CAN communication) and comparison with legacy AUTOSAR-based implementation [116]
	E-Gas	(Custom)	Custom hypervisor for integrating real-time and safety partitions that uses the automotive E-Gas reference concept as case study [154]
	AD	Jailhouse, KVM	Hypervisor-based software stack architecture proposal for AD systems [32, 43]. For instance, for Apollo AD framework and using heterogeneous guest OSs (e.g., Linux, Erika) [32]
		Xen	Reference safety architecture (ASILC) that extends E-Gas layered monitoring concept for autonomous driving solutions, supporting the integration of multiple mixed-criticality software partitions with associated RTOS/OS and middlewares in HPC devices [36]
Avionics	Generic	seL4, LynxSecure, Xen	Generic aspects and arguments for avionics safety systems: seL4 [224], Xen and LynxSecure on Zynq Ultrascale MPSoC [227]
		seL4	Describes three software partitioning approaches for avionics (adaptation of the RTOS, hypervisor, microkernel) [226]
	Multicopter	Jailhouse	Hardware resources partitioning and Linux-based Jailhouse hypervisor-based safety mixed-criticality software partitioning [184]
		-	IMA approach for the development of reduced SWaP multicopter that uses a partitioning hypervisor approach with static hardware partitioning among RTOS (Quest) and OS (Linux) virtualized sandboxes [75]
Railway	Railway signaling	XtratuM	Safety-concept of a railway signaling object-controller safety-critical system (SIL4) with safety and low-power requirements, based on multi-core device and XtratuM hypervisor [37].
		PikeOS	Safety-concept of a railway signaling system that integrates software applications of different criticality (e.g., SIL4) on a multi-code device and PikeOS hypervisor-based solution [13].
Space	Generic	XtratuM	Evaluation of miscellaneous space applications and strategies with XtratuM: hypervisor-based fault-tolerance strategy [45], overhead analysis [112], the impact of multi-core device [166].
		XtratuM	Reference architecture for the deployment of safety-critical Java applications on a given software partition with PaTiKLe OS [80].
		PikeOS, XtratuM	Benchmark analysis and comparison of XtratuM and PikeOS hypervisors using LEON3 and LEON4 type space processors [152].
		(Unnamed)	Overhead performance improvements for a dual-core LEON4 space architecture with a fully virtualized type II micro-hypervisor for space applications (host: Linux, guest: RTEMS) [105].
Generic	Generic	XtratuM	Generic dependable and secure architecture proposal. It also provides an IPC temporal benchmark for supported communication channel types [219]
		Rodosvisor	Generic analysis of full-virtualization overhead (temporal, footprint) on a low-end embedded hardware platform [48]
Industry	Wind Turbine	XtratuM	Safety-concept [168], temporal independence validation [120] and safety certification strategy [167] for wind-turbine safety protection systems based on virtualization and multi-core devices
	Machine ICS	Jailhouse	Machine control case-study that integrates real-time and best-effort partitions [25]

7.2 Industrial Domain

The industrial domain has heterogeneous applications ranging from **Programmable Logic Controller (PLC)** based ICSs installed in manufacturing plants to offshore wind turbines. Nonetheless, with different levels of importance, they share common technical needs such as the need to continue integrating more software functionalities of different criticality, increasing computing performance, and reducing the number, cost and SWaP of integrated embedded systems [25, 52, 167, 168, 204]. For instance, Simó et al. [204] provide an overview of the role of mixed-criticality technology in Industry 4.0, with the digitalization (e.g., IIoT) and integration of new technologies (e.g., big data, ML) in ICSs initially developed to execute dependable real-time dependable control functions, and emphasizes the pivotal role of virtualization solutions to support this integration of mixed-criticality functions in HPC computing devices. Additionally, the integration of **Information**

Technology (IT) and Operation Technology (OT) requirements in ICSs requires safety and security considerations [136]. In this context, as summarized in Table 3, diverse architectures use diverse virtualization solutions to develop mixed-criticality applications combining real-time and non-real-time software partitions and safety-critical and non-safety-related partitions on HPC devices [25, 167, 168]. Finally, due to the absence of industry-level standards such as ARINC 653 or AUTOSAR and a higher diversity of applications, a wider variety of virtualization solutions can be used in industrial domain systems except for the ones requiring compliance with functional safety requirements (see Table 2).

8 Conclusion and Future Research Directions

Virtualization technology in combination with embedded HPC devices can support the reconciliation of two contradictory challenges facing next-generation transportation and industrial systems: a growing demand to integrate more (e.g., software applications of mixed-criticality, computational performance, software stacks, OSs) with the need to contain or reduce the cost, number and SWaP of integrated embedded systems. As summarized in Table 2, virtualization technology for dependable embedded systems has evolved into a heterogeneous variety of virtualization solutions (hypervisor, microkernel, separation kernel) that support a limited set of computing architectures (e.g., ARM, x86, LEON, RISC-V), diverse OSs and scheduling algorithms, and only a subset of the solutions comply with domain-specific safety standards. Nowadays, as summarized in Table 1, virtualization technology already supports the development of dependable embedded systems in transportation and industrial domains, including safety-critical systems, and it is continuously evolving and subject to evaluations with domain-specific cases-studies as summarized in Table 3.

Providing virtualization and encapsulation of computing resources with a low and predictable computing overhead, spatial/temporal isolation guarantees and dependability attributes is a past, present and future challenge for the developers of hardware-assisted and software virtualization solutions. The evolution of virtualization technology over the last decades has been enormous: from hypervisors that accommodate paravirtualized environments in single-core embedded devices to multi-core device support where the hypervisor provides execution environments with one or more virtual CPUs, and in recent years, hardware-assisted virtualization technology with a hypervisor mode that allows the virtualization layer to penetrate the hardware and execution environments to run on virtual hardware that fully mimics the native one. This evolution will continue in the future as the features, hardware-assisted virtualization and resources provided by HPC devices continue to increase and evolve [169, 171]. At the hypervisor level, the more services the hardware-assisted virtualization offers, the fewer functions the hypervisor will need to implement, but the more dependability-related non-functional features are required for managing aspects such as security, reliability, robustness, HM diagnostics management, integrity and safety.

Additionally, as the complexity and features provided by HPC devices continue to increase [169, 171], the diversity and amount of computing resources to be virtualized will continue to increase (e.g., ML accelerator). Furthermore, this will also lead to amplifying current real-time guarantee challenges due to a potential increase of temporal interference in HPC devices and high difficulty (if feasible) to provide trustworthy WCET estimates [169, 171]. So, the need for temporal observability mechanisms (e.g., PMU), mitigation techniques deployed at hardware/software virtualization solutions (e.g., partition scheduling) and software architecture level will continue to be important to meet firm and hard deadlines.

Providing dependable virtualization solutions that support the cost-efficient development of dependable embedded systems, including the highest criticality systems, is a present and future research challenge for both virtualization solutions developers and application-specific software architects/developers. These applications can be developed by different software teams developing

software partitions of mixed-criticality leading to software engineering and toolset challenges to support their development and integration [85]. Additionally, as heterogeneous distributed embedded systems might integrate one or more virtualization solutions, providing a single view of multiple distributed virtualization solutions with distributed partitions and their local/remote IPC is a future technical challenge to be highlighted.

Finally, virtualization technology evolution will also adapt to each domain's specific features and characteristics, leading to specialized solutions, architectures, patterns and supported standards. For instance, in the avionics domain, the integration of dynamic virtualization in heterogeneous computing resources, such as CPUs, GPUs and FPGAs, along with the influence of **Artificial Intelligence (AI)**, has considerably enhanced the capabilities and efficiency of avionic systems. However, these advancements necessitate robust strategies for design determinism and safety mitigation in the face of dynamic reconfiguration, particularly for accurately determining WCET in real-time systems. With continued research and innovation, we can expect avionic systems to evolve toward greater performance, safety and adaptability.

References

- [1] 2005. RTCA DO-297. Integrated Modular Avionics (IMA) Development Guidance and Certification Considerations. (2005).
- [2] 2021. PikeOS product overview. Retrieved from https://www.sysgo.com/fileadmin/user_upload/data/flyers_brochures/SYSGO_PikeOS_Product_Overview.pdf. (2021). [Online; accessed 29-October-2021].
- [3] 2021. White paper: Solving Multicore Interference for Safety Critical Applications. Retrieved from https://www.ghs.com/download/whitepapers/GHS_multicore_interference.pdf. (2021).
- [4] 2022. COQOS. Retrieved from <https://www.opensynergy.com/automotive-hypervisor/>. (2022). [Online; accessed 11-October-2022].
- [5] 2022. Kernel Virtual Machine (KVM). Retrieved from <https://www.linux-kvm.org/>. (2022). [Online; accessed 11-October-2022].
- [6] 2022. LWHVR product overview. Retrieved from <https://www.etas.com/en/products/rta-lwhvr.php>. (2022). [Online; accessed 11-Oct.-2022].
- [7] 2022. RTS Hypervisor. Retrieved from <https://www.real-time-systems.com/products/rts-hypervisor.html>. (2022). [Online; accessed 18-October-2022].
- [8] 2022. Trusted Software for Embedded. Retrieved from <https://www.ghs.com/products.html>. (2022). [Online; accessed 2-November-2022].
- [9] 2023. EB corbos hypervisor. Retrieved from <https://www.elektrobit.com>. (2023). [Online; accessed 15-February-2023].
- [10] Andreas Abel et al. 2013. Impact of resource sharing on performance and performance prediction: A survey. In *CONCUR 2013—Concurrency Theory*. Springer Berlin, Berlin, 25–43.
- [11] J. Abella et al. 2015. WCET analysis methods: Pitfalls and challenges on their trustworthiness. In *10th IEEE International Symposium on Industrial Embedded Systems (SIES)*. DOI : <https://doi.org/10.1109/SIES.2015.7185039>
- [12] L Abeni and D Faggioli. 2020. Using Xen and KVM as real-time hypervisors. *J. Syst. Archit.* 106 (2020). DOI : <https://doi.org/10.1016/j.sysarc.2020.101709>
- [13] Irune Agirre et al. 2015. A safety concept for a railway mixed-criticality embedded system based on multicore partitioning. In *IEEE Internat. Conf. on Computer and Inf. Technol.; Ubiquitous Comput. and Commun.; Dependable, Autonomic and Secure Comput.; Pervasive Intell. and Comput. (CIT/IUCC/DASC/PICOM)*. 1780–1787. DOI : <https://doi.org/10.1109/CIT/IUCC/DASC/PICOM.2015.268>
- [14] Eyad Alkassar et al. 2010. Automated verification of a small hypervisor. In *Third Internat. Conf. on Verified Software: Theories, Tools, Experiments (VSTTE)*, Vol. LNCS 6217. Springer-Verlag, 40–54.
- [15] Sara Alonso, Jesús Lázaro, Jaime Jiménez, Unai Bidarte, and Leire Muguiru. 2021. Evaluating latency in multiprocessing embedded systems for the smart grid. *Energies* 14, 11 (2021), 3322.
- [16] ARINC 1996. *Avionics Application Software Standard Interface (ARINC-653)*. ARINC. Airlines electronic Eng. committee.
- [17] Arm. 2017. *ARM CoreLink QoS-400 Network Interconnect Advanced Quality of Service Supplement to ARM CoreLink NIC-400 Network Interconnect Technical Reference Manual*.
- [18] Arm. 2017. *ARM CoreLink QVN-400 Network Interconnect Advanced Quality of Service using Virtual Networks Supplement to ARM CoreLink NIC-400 Network Interconnect Technical Reference Manual*.
- [19] ARM. 2022. Architecture Reference Manual Supplement Memory System Resource Partitioning and Monitoring (MPAM), for A-profile architecture. (2022). ARM DDI 0598D.a (ID042622).

- [20] François Armand and Michel Gien. 2009. A practical look at micro-kernels and virtual machine monitors. In *6th IEEE Consumer Communications and Networking Conf.* IEEE, 1–7.
- [21] A. Avizienis, J. C. Laprie, B. Randell, and C. Landwehr. 2004. Basic concepts and taxonomy of dependable and secure computing. In *IEEE Trans. on Dependable and Secure Comput.*, Vol. 1. 11–33.
- [22] Ayoosh Bansal, Rohan Tabish, Giovanni Gracioli, Renato Mancuso, Rodolfo Pellizzoni, and Marco Caccamo. 2018. Evaluating memory subsystem of configurable heterogeneous MPSoC. (2018). Retrieved from <https://open.bu.edu/handle/2144/37758>
- [23] Paul Barham, Boris Dragovic, Keir Fraser, Steven Hand, Tim Harris, Alex Ho, Rolf Neugebauer, Ian Pratt, and Andrew Warfield. 2003. Xen and the art of virtualization. *ACM SIGOPS Operating Systems Review* 37, 5 (2003), 164–177.
- [24] Gilles Barthe, Gustavo Betarte, Juan Diego Campo, and Carlos Luna. 2011. Formally verifying isolation and availability in an idealized model of virtualization (*FM 2011: Formal Methods*). Springer Berlin, 231–245.
- [25] David Barton, Philipp Gönninger, Florian Schade, Christopher Ehrmann, Jürgen Becker, and Jürgen Fleischer. 2019. Modular smart controller for Industry 4.0 functions in machine tools. *Procedia CIRP* 81 (2019), 1331–1336. DOI : <https://doi.org/10.1016/j.procir.2019.04.022>
- [26] Christoph Baumann et al. 2009. *Formal Verification of a Microkernel Used in Dependable Software Systems*. Lecture Notes in Computer Science (LNCS), Vol. LNCS 5775. Springer, Berlin / Heidelberg, 187–200. DOI : https://doi.org/10.1007/978-3-642-04468-7_16
- [27] Christoph Baumann et al. 2011. Proving memory separation in a microkernel by code level verification. In *1st Internat. Workshop on Architectures and Applications for Mixed-Criticality Systems (AMICS)*.
- [28] Michael Bechtel and Heechul Yun. 2019. Denial-of-service attacks on shared cache in multicore: Analysis and prevention. In *IEEE Real-Time and Embedded Technology and Applications Symp. (RTAS)*. 357–367.
- [29] Matthias Beckett et al. 2014. Sufficient temporal independence and improved interrupt latencies in a real-time hypervisor. In *51st Annual Design Automation Conf. on Design Automation Conf.* ACM, 2593222, 1–6. DOI : <https://doi.org/10.1145/2593069.2593222>
- [30] Matthias Beckett and Rolf Ernst. 2015. Designing time partitions for real-time hypervisor with sufficient temporal independence. In *52nd ACM/EDAC/IEEE Design Automation Conf. (DAC)*. IEEE, 1–6.
- [31] Y. B. Bekele and D. B. Limbrick. 2022. Evaluating the impact of hardware faults on program execution in a microkernel environment. In *IEEE Internat. Symp. on Hardware Oriented Security and Trust (HOST)*. 149–152. DOI : <https://doi.org/10.1109/HOST54066.2022.9840063>
- [32] L. Belluardo et al. 2021. A multi-domain software architecture for safe and secure autonomous driving. In *IEEE 27th Internat. Conf. on Embedded and Real-Time Computing Systems and Applications (RTCSA)*. 73–82. DOI : <https://doi.org/10.1109/RTCSA52859.2021.00017>
- [33] Muli Ben-Yehuda, Jimi Xenidis, Michal Ostrowski, Karl Rister, Alexis Bruemmer, and Leendert Van Doorn. 2007. The price of safety: Evaluating IOMMU performance. In *The Ottawa Linux Symp.* 9–20.
- [34] Ram Chandra Bhushan and Dharmendra K. Yadav. 2019. Modelling and formally verifying Intel VT-x: Hardware assistance for processors running virtualization platforms. *International Journal of Engineering and Advanced Technology (IJEAT)* 8, 4 (2019), 241–247.
- [35] Ram Chandra Bhushan and Dharmendra K. Yadav. 2020. Verification of Virtual Machine Architecture in a Hypervisor through Model Checking. *Procedia Computer Science* 167 (2020), 67–74. DOI : <https://doi.org/10.1016/j.procs.2020.03.183>
- [36] Tjerk Bijlsma et al. 2020. A distributed safety mechanism using middleware and hypervisors for autonomous vehicles. In *Design, Automation & Test in Europe Conf. & Exhibition (DATE)*. IEEE, 1175–1180.
- [37] Ainara Bilbao et al. 2017. A railway safety and security concept for low-power mixed-criticality systems. In *IEEE 15th Int. Conf. on Industrial Informatics (INDIN)*. 59–64.
- [38] A. Biondi et al. 2021. SPHERE: A Multi-SoC Architecture for Next-Generation Cyber-Physical Systems Based on Heterogeneous Platforms. *IEEE Access* 9 (2021), 75446–75459. DOI : <https://doi.org/10.1109/ACCESS.2021.3080842>
- [39] Alessandro Biondi, Mauro Marinoni, Giorgio Buttazzo, Claudio Scordino, and Paolo Gai. 2018. Challenges in virtualizing safety-critical cyber-physical systems. In *Proceedings of Embedded World Conf.* 1–5.
- [40] B. Blackham, Y. Shi, S. Chattopadhyay, A. Roychoudhury, and G. Heiser. 2011. Timing Analysis of a Protected Operating System Kernel. In *IEEE 32nd Real-Time Systems Symp.* 339–348. DOI : <https://doi.org/10.1109/RTSS.2011.38>
- [41] N. Borgioli et al. 2022. An I/O Virtualization Framework With I/O-Related Memory Contention Control for Real-Time Systems. *IEEE Trans. on Computer-Aided Design of Integrated Circuits and Systems* 41, 11 (2022), 4469–4480. DOI : <https://doi.org/10.1109/TCAD.2022.3202434>
- [42] Andrew Boyton et al. 2013. *Formally verified system initialisation (Formal Methods and Software Engineering)*. Springer Berlin, 70–85.
- [43] P. Burgio et al. 2016. A software stack for next-generation automotive systems on many-core heterogeneous platforms. In *Euromicro Conf. on Digital System Design (DSD)*. 55–59. DOI : <https://doi.org/10.1109/DSD.2016.84>

- [44] Flavia Caforio, Pierpaolo Iannicelli, Michele Paolino, and Daniel Raho. 2021. VOSySmonitoRV: A mixed-criticality solution on Linux-capable RISC-V platforms. In *10th Mediterranean Conf. on Embedded Computing (MECO)*. IEEE, 1–4.
- [45] Salvatore Campagna et al. 2010. Hypervisor-based virtual hardware for fault tolerance in COTS processors targeting space applications. In *IEEE 25th Internat. Symp. on Defect and Fault Tolerance in VLSI Systems*. IEEE, 44–51.
- [46] Elena Carrascosa, Javier Coronel, Miguel Masmano, Patricia Balbastre, and Alfons Crespo. 2014. XtratuM hypervisor redesign for LEON4 multicore processor. *ACM SIGBED Review* 11, 2 (2014), 27–31.
- [47] E. Carrascosa, M. Masmano, P. Balbastre, A. Crespo, and J. Galizzi. 2014. Multicore software architectures on virtualized partitioned systems. In *Data Systems In Aerospace (DASIA)*, L. Ouwehand (Ed.), Vol. 725. 27. Retrieved from <https://ui.adsabs.harvard.edu/abs/2014ESASP.725E..27C>
- [48] A. Carvalho et al. 2016. Full virtualization on low-end hardware: A case study. In *IECON 2016 - 42nd Annual Conference of the IEEE Industrial Electronics Society*. 4784–4789. DOI: <https://doi.org/10.1109/IECON.2016.7794064>
- [49] Daniel Casini, Alessandro Biondi, Giorgiomaia Cicero, and Giorgio Buttazzo. 2021. Latency analysis of I/O virtualization techniques in hypervisor-based real-time systems. In *Proc. IEEE Real-Time Embedded Technol. Appl. Symp. (RTAS)*.
- [50] Daniel Casini, Alessandro Biondi, Giorgiomaia Cicero, and Giorgio C. Buttazzo. 2021. Latency analysis of I/O virtualization techniques in hypervisor-based real-time systems. In *RTAS*. IEEE, 306–319.
- [51] David Chisnall. 2013. *The Definitive Guide to the Xen Hypervisor*. Prentice Hall Press.
- [52] Marcello Cinque, Domenico Cotroneo, Luigi De Simone, and Stefano Rosiello. 2022. Virtualizing mixed-criticality systems: A survey on industrial trends and issues. In *Future Generation Computer Systems*, Vol. 129. 315–330. DOI: <https://doi.org/10.1016/j.future.2021.12.002>
- [53] M. Cinque, L. De Simone, and A. Marchetta. 2022. Certify the uncertified: Towards assessment of virtualization for mixed-criticality in the automotive domain. In *52nd Annual IEEE/IFIP Internat. Conf. on Dependable Systems and Networks Workshops (DSN-W)*. 8–11. DOI: <https://doi.org/10.1109/DSN-W54100.2022.00012>
- [54] Marcello Cinque and Antonio Pecchia. 2017. On the injection of hardware faults in virtualized multicore systems. *J. Parallel and Distrib. Comput.* 106 (2017), 50–61. DOI: <https://doi.org/10.1016/j.jpdc.2017.03.004>
- [55] Alfons Crespo, Alejandro Alonso, Marga Marcos, A Juan, and Patricia Balbastre. 2014. Mixed criticality in control systems. *IFAC Proceedings Volumes* 47, 3 (2014), 12261–12271.
- [56] A. Crespo, M. Masmano, J. Coronel, S. Peiró, P. Balbastre, and J. Simó. 2014. Multicore partitioned systems based on hypervisor. *IFAC Proceedings Volumes* 47, 3 (2014), 12293–12298. DOI: <https://doi.org/10.3182/20140824-6-ZA-1003.02410>
- [57] A. Crespo, I. Ripoll, and M. Masmano. 2010. Partitioned embedded architecture based on hypervisor: The XtratuM approach. In *European Dependable Computing Conf. (EDCC)*. 67–72. DOI: <https://doi.org/10.1109/EDCC.2010.18>
- [58] Alfons Crespo, I. Ripoll, M. Masmano, P. Arberet, and J. J. Metge. 2009. XtratuM: An open source hypervisor for TSP embedded systems in aerospace. In *Data Systems In Aerospace (DASIA)*.
- [59] Tommaso Cucinotta, Gaetano Anastasi, and Luca Abeni. 2009. Respecting temporal constraints in virtualised services. In *33rd Annual IEEE Internat. Computer Software and Applications Conf.*, Vol. 2. 73–78. DOI: <https://doi.org/10.1109/COMPSAC.2009.118>
- [60] Christoffer Dall, Shih-Wei Li, Jin Tack Lim, and Jason Nieh. 2018. ARM virtualization: Performance and architectural implications. *SIGOPS Oper. Syst. Rev.* 52, 1 (2018), 45–56. DOI: <https://doi.org/10.1145/3273982.3273987>
- [61] Jakob Danielsson, Tiberiu Seceleanu, Marcus Jägemar, Moris Behnam, and Mikael Sjödin. 2019. Testing performance-isolation in multi-core systems. In *IEEE 43rd Annual Computer Software and Applications Conf. (COMPSAC)*, Vol. 1. 604–609.
- [62] Matthew Danish, Ye Li, and Richard West. 2011. Virtual-CPU scheduling in the quest operating system. In *17th IEEE Real-Time and Embedded Technology and Applications Symp.* 169–179. DOI: <https://doi.org/10.1109/RTAS.2011.24>
- [63] Dakshina Dasari et al. 2020. Applying Reservation-based Scheduling to a μ C-based Hypervisor: An industrial case study. In *Design, Automation & Test in Europe Conf. & Exhibition (DATE)*. 987–990.
- [64] DDC-I. 2022. Deos, a time and space partitioned, multi-core enabled, RTOS verified to DO-178C ED-12C DAL A. (2022). Retrieved from https://www.ddci.com/products_deos_do_178c_arinc_653/
- [65] Jiaqing Du, Nipun Sehrawat, and Willy Zwaenepoel. 2010. Performance profiling in a virtualized environment. *2nd USENIX Workshop on Hot Topics in Cloud Computing*.
- [66] Julien Durand et al. 2019. Statistical analysis for shared resources effects with multi-core real-time systems. In *IEEE 13th Int. Symp. on Embedded Multicore/Many-core Systems-on-Chip (MCSoc)*. 362–371. DOI: <https://doi.org/10.1109/MCSoc.2019.00058>
- [67] ECSS. 2013. *Standard ECSS-E-40: Space Software: Engineering*. Technical Report. ECSS.
- [68] Mohamed El Mehdi Aichouc. 2014. *Evaluation of a Multiple-Criticality Real-Time Virtual Machine System and Configuration of an RTOS's Resources Allocation Techniques*. Thesis. Retrieved from <https://theses.hal.science/tel-01127450/document>

- [69] S. Esposito, S. Avramenko, and M. Violante. 2016. On the consolidation of mixed criticalities applications on multicore architectures. In *17th Latin-American Test Symp. (LATS)*. 57–62. DOI : <https://doi.org/10.1109/LATW.2016.7483340>
- [70] European Union Aviation Safety Agency (EASA). 2023. *EASA Artificial Intelligence Roadmap 2.0*. Technical Report.
- [71] J. C. Fabre, M. Rodriguez, J. Arlat, and J. M. Sizun. 2000. Building dependable COTS microkernel-based systems using MAFALDA. In *Pacific Rim Internat. Symp. on Dependable Computing*. 85–92. DOI : <https://doi.org/10.1109/PRDC.2000.897288>
- [72] J. C. Fabre, F. Salles, M. R. Moreno, and J. Arlat. 1999. Assessment of COTS microkernels by fault injection. In *Dependable Computing for Critical Applications* 7. 25–44. DOI : <https://doi.org/10.1109/DCFTS.1999.814288>
- [73] Maher Fakih et al. 2017. SAFEPOWER project: Architecture for safe and power-efficient mixed-criticality systems. *Microprocessors and Microsystems* 52 (2017), 89–105. DOI : <https://doi.org/10.1016/j.micpro.2017.05.016>
- [74] Zhang Fan and Wang Xiaopeng. 2013. A model of microkernel based on spatial-temporal isolation in Haskell. In *Internat. Conf. on Computer Sciences and Applications (CSA)*. 564–569. DOI : <https://doi.org/10.1109/CSA.2013.137>
- [75] A. Farrukh and R. West. 2022. FLYOS: Integrated modular avionics for autonomous multicopters. In *IEEE 28th Real-Time and Embedded Technology and Applications Symp. (RTAS)*. 68–81. DOI : <https://doi.org/10.1109/RTAS54340.2022.00014>
- [76] Matthias Gerlach, Stephan Weissleder, and Robert Hilbrich. 2011. Benefits of reusing avionics components for ISO 26262 developments in cars. In *15th Internat. Congress on Electronic Systems for Motor Vehicles*.
- [77] Jeremy Giesen, Enrico Mezzetti, Jaume Abella, and Francisco J. Cazorla. 2021. PRL: Standardizing performance monitoring library for high-integrity real-time systems. In *ICCD*. IEEE, 344–348.
- [78] Katharina Gilles, Stefan Groesbrink, Daniel Baldin, and Timo Kerstan. 2013. Proteus hypervisor: Full virtualization and paravirtualization for multi-core embedded systems. In *Internat. Embedded Systems Symp.* Springer, 293–305.
- [79] GMV. 2023. AIR User Manual. Retrieved from <https://gmvdrive.gmv.com/index.php/s/eScXCAYbbecmT9b>. (2023). Accessed: 2023-05-09.
- [80] Apolinar González et al. 2013. A hypervisor based platform to support real-time safety critical embedded java applications. *Computer Systems Science and Engineering* 28 (2013), 157–168.
- [81] Zonghua Gu and Qingling Zhao. 2012. A State-of-the-art survey on real-time issues in embedded systems virtualization. *Journal of Software Engineering and Applications* 5, 1 (2012), 277–290. DOI : <https://doi.org/10.4236/jsea.2012.54033>
- [82] H. Haidong, L. Wei, Z. Tao, H. Xuesong, B. Chenyang, and Z. Hongpeng. 2021. TMAC: An active defense architecture adapted to microkernel operating system for power ICS. In *China Automation Congress (CAC)*. 5195–5199. DOI : <https://doi.org/10.1109/CAC53003.2021.9727287>
- [83] Etienne Hamelin, M. Ait Hmid, Amine Naji, and Yves Mouafo-Tchinda. 2020. Selection and evaluation of an embedded hypervisor: Application to an automotive platform. In *European Congress of Embedded Real Time Software and Systems*.
- [84] Brahim Hamid and Jon Perez. 2016. Supporting pattern-based dependability engineering via model-driven development: Approach, tool-support and empirical validation. *Systems and Software, Journal on* 122 (2016), 239–279. DOI : <https://doi.org/10.1016/j.jss.2016.09.027>
- [85] Ahmadian Hamidreza, Roman Obermaisser, and Jon Perez. 2018. *Distributed Real-Time Architecture for Mixed-Criticality Systems*. CRC Press.
- [86] David Haworth. 2012. An AUTOSAR-compatible microkernel for systems with safety-relevant components (*Herausforderungen durch Echtzeitbetrieb*). Springer Berlin, 11–20.
- [87] David Haworth, Tobias Jordan, Alexander Mattausch, and Alexander Much. 2012. Freedom from interference for AUTOSAR-based ECUs: A partitioned AUTOSAR stack. In *Automotive - Safety & Security*.
- [88] Gernot Heiser. 2020. The seL4 Microkernel - An Introduction. Retrieved from <https://sel4.systems/About/seL4-whitepaper.pdf>. (2020). [Online; accessed 10-January-2023].
- [89] Gernot Heiser and Ben Leslie. 2010. The OKL4 microvisor: Convergence point of microkernels and hypervisors. In *1st ACM SIGCOMM Asia-Pacific Workshop on Systems*. 19–24. DOI : <https://doi.org/10.1145/1851276.1851282>
- [90] G. Heiser, T. Murray, and G. Klein. 2012. It's time for trustworthy systems. *IEEE Security & Privacy* 10, 2 (2012), 67–70. DOI : <https://doi.org/10.1109/MSP.2012.41>
- [91] Carles Hernandez et al. 2020. SELENE: Self-monitored dependable platform for high-performance safety-critical systems. In *23rd Euromicro Conf. on Digital System Design (DSD)*. IEEE, 370–377.
- [92] Tobias Holstein and Joachim Wietzke. 2015. *Contradiction of Separation through Virtualization and Intercommunication*. 483–490. DOI : <https://doi.org/10.1515/9783110443905-069>
- [93] IEC 2010. IEC 61508(-1/7): Functional safety of electrical/electronic/programmable electronic safety-related systems. (2010).
- [94] Intel. 2019. Intel Resource Director Technology (Intel RDT) on 2nd Generation Intel Xeon Scalable Processors Reference Manual, Rev. 1.0. (2019).
- [95] Intel. 2022. Profile Targets in the Hyper-V* Environment. Retrieved from <https://www.intel.com/content/www/us/en/develop/documentation/vtune-help/top/set-up-analysis-target/on-virtual-machine/profiling-on-hyper-v.html>. (2022).
- [96] ISO 2018. ISO 26262(-1/11) Road vehicles - Functional safety. (2018).

- [97] Boguslaw Jablkowski and Olaf Spinczyk. 2015. CPS-Xen: A virtual execution environment for cyber-physical applications. In *Architecture of Computing Systems (ARCS)*. 108–119.
- [98] Richard Jaenicke. 2020. What's the Difference Between an Embedded Hypervisor and Separation Microkernel with Virtualization? *Electronic Design* (February 2020).
- [99] Z. Jiang et al. 2021. I/O-GUARD: Hardware/Software Co-Design for I/O Virtualization with Guaranteed Real-time Performance. In *58th ACM/IEEE Design Automation Conf. (DAC)*. 1159–1164. DOI : <https://doi.org/10.1109/DAC18074.2021.9586156>
- [100] Zhe Jiang and Neil Audsley. 2017. VDCD: The virtualized complicated device controller. In *29th Euromicro Conference on Real-Time Systems (ECRTS) (Leibniz International Proceedings in Informatics (LIPIcs))*, Vol. 76. 5:1–5:21.
- [101] Zhe Jiang, Neil Audsley, Pan Dong, Nan Guan, Xiaotian Dai, and Lifeng Wei. 2019. MCS-IOV: Real-time I/O virtualization for mixed-criticality systems. In *IEEE Real-Time Systems Symposium (RTSS)*. 326–338. DOI : <https://doi.org/10.1109/RTSS46320.2019.00037>
- [102] Z. Jiang, N. C. Audsley, and P. Dong. 2018. BlueVisor: A scalable real-time hardware hypervisor for many-core embedded systems. In *IEEE Real-Time and Embedded Technology and Applications Symp. (RTAS)*. 75–84. DOI : <https://doi.org/10.1109/RTAS.2018.00013>
- [103] Z. Jiang, R. Wei, P. Dong, Y. Zhuang, N. Audsley, and I. Gray. 2021. BlueVisor: Time-predictable hardware hypervisor for many-core embedded systems. *IEEE Trans. on Computers* (2021), 1–1. DOI : <https://doi.org/10.1109/TC.2021.3125226>
- [104] Payal Jingare and Priyanka Sorte. 2021. A survey on virtualization and hypervisor-based technology in cloud computing environment. *Internat. Journal of Scientific & Engineering Research* 12, 3 (2021).
- [105] Hyunwoo Joe, Hyeona Jeong, Youngil Yoon, Hyungshin Kim, Sanghyun Han, and Hyun-Wook Jin. 2012. Full virtualizing micro hypervisor for spacecraft flight computer. In *IEEE/AIAA 31st Digital Avionics Systems Conf. (DASC)*. IEEE, 6C5–1–6C5–9.
- [106] N. Jomaa, D. Nowak, G. Grimaud, and S. Hym. 2016. Formal proof of dynamic memory isolation based on MMU. In *10th Internat. Symp. on Theoretical Aspects of Software Engineering (TASE)*. 73–80. DOI : <https://doi.org/10.1109/TASE.2016.28>
- [107] Robert Kaiser. 2008. Alternatives for scheduling virtual machines in real-time embedded systems. In *1st Workshop on Isolation and Integration in Embedded Systems (IIES)*. Association for Computing Machinery, New York, NY, USA, 5–10. DOI : <https://doi.org/10.1145/1435458.1435460>
- [108] Robert Kaiser and Stephan Wagner. 2007. Evolution of the PikeOS Microkernel. In *First Internat. Workshop on MicroKernels for Embedded Systems. (MIKES)*. Sydney, Australia.
- [109] Q. Kang, C. Yuan, X. Wei, Y. Gao, and L. Wang. 2016. A user-level approach for ARINC 653 temporal partitioning in seL4. In *Internat. Symp. on System and Software Reliability (ISSSR)*. 106–110. DOI : <https://doi.org/10.1109/ISSSR.2016.025>
- [110] S. Karthik et al. 2018. Hypervisor based approach for integrated cockpit solutions. In *IEEE Int. Conf. on Consumer Electronics - Berlin (ICCE-Berlin)*. 1–6. DOI : <https://doi.org/10.1109/ICCE-Berlin.2018.8576222>
- [111] SooYoung Kim, Hyunwoo Joe, Dongwook Kang, Jin-Ah Shin, Vincent Dupre, Taeho Kim, and Chaedeok Lim. Device dedication in virtualized embedded systems (*Advances in Computer Science and Ubiquitous Computing*). Springer Singapore, 295–299.
- [112] Sun-Wook Kim, Bum-Soo Yoo, Jae-Yeop Jeong, and Jong-Wook Choi. 2020. Overhead analysis of XtratuM for space in SMP environment. *IEMEK Journal of Embedded Systems and Applications* 15, 4 (2020), 177–187.
- [113] D. Kleidermacher and M. Wolf. 2008. MILS virtualization for Integrated Modular Avionics. In *IEEE/AIAA 27th Digital Avionics Systems Conference*. 1.C.3–1–1.C.3–8. DOI : <https://doi.org/10.1109/DASC.2008.4702759>
- [114] Gerwin Klein et al. 2010. seL4: Formal Verification of an Operating-System Kernel. *Commun. ACM* 53, 6 (2010), 107–115. DOI : <https://doi.org/10.1145/1743546.1743574>
- [115] T. Kloda et al. 2019. Deterministic memory hierarchy and virtualization for modern multi-core embedded systems. In *IEEE Real-Time and Embedded Technology and Applications Symp. (RTAS)*. 1–14. DOI : <https://doi.org/10.1109/RTAS.2019.00009>
- [116] Andre Kohn et al. 2017. Timing analysis for hypervisor-based I/O virtualization in safety-related automotive systems. *SAE Internat. Journal of Passenger Cars-Electronic and Electrical Systems* 10, 2017-01-1621 (2017), 368–379.
- [117] L4Re.org. 2023. L4Re microkernel. <https://l4re.org>. (2023). Online; accessed 15-February-2023.
- [118] Kai Lampka, Joel Thurlby, Adam Lackorzynski, and Marcus Hähnel. 2022. Safety certification with the open source microkernel-based operating system L4Re (*Computer Safety, Reliability, and Security*). Springer Internat. Publishing, 31–45.
- [119] John R. Lange, Nicholas Gordon, and Brian Gaines. 2021. Low overhead security isolation using lightweight kernels and TEEs. In *SC Workshops Supplementary Proceedings (SCWS)*. 42–49. DOI : <https://doi.org/10.1109/SCWS55283.2021.00016>
- [120] Asier Larrucea et al. 2015. Temporal independence validation of an IEC-61508 compliant mixed-criticality system based on multicore partitioning. In *Forum on Specification and Design Languages (FDL)*. 8.
- [121] Asier Larrucea, Imanol Martinez, Hamidreza Ahmadian, Roman Obermaisser, Vicent Brocal, Salvador Peiró, and Jon Perez. 2016. DREAMS: Cross-domain mixed-criticality patterns. In *Mixed-Criticality Workshop on Real Time System Symp. (RTSS)*. 8.

- [122] Asier Larrucea, Imanol Martinez, Vicent Brocal, Hamidreza Ahmadian, Salvador Peir'o, Roman Obermaisser, and Jon Perez. 2017. Reusable generic design patterns for mixed-criticality systems based on DREAMS. *Microprocessors and Microsystems* 54 (2017), 35–46. DOI : <https://doi.org/10.1016/j.micpro.2017.06.017>
- [123] Asier Larrucea, Jon Perez, Irune Agirre, Vicent Brocal, and Roman Obermaisser. 2015. A modular safety case for an IEC-61508 compliant generic hypervisor. In *Euromicro Conf. on Digital System Design (DSD)*. CPS, 4. DOI : <https://doi.org/10.1109/dsd.2015.27>
- [124] T. Y. Lee, H. R. Seo, and D. R. Shin. 2010. Analysis of component model by kernel structures. In *Internat. Conf. on Electronics and Information Engineering*, Vol. 2. V2–45–V2–48. DOI : <https://doi.org/10.1109/ICEIE.2010.5559712>
- [125] D. Conte de Leon and S. Shrestha. 2016. Requirements are the new code. In *IEEE 40th Annual Computer Software and Applications Conf. (COMPSAC)*, Vol. 1. 691–694. DOI : <https://doi.org/10.1109/COMPSAC.2016.265>
- [126] Shih-Wei Li, Xupeng Li, Ronghui Gu, Jason Nieh, and John Zhuang Hui. 2021. Formally verified memory protection for a commodity multiprocessor hypervisor. In *30th USENIX Security Symposium (USENIX Security 21)*. 3953–3970.
- [127] Shih-Wei Li, Xupeng Li, Ronghui Gu, Jason Nieh, and John Zhuang Hui. 2021. A secure and formally verified Linux KVM hypervisor. In *IEEE Symposium on Security and Privacy (SP)*. IEEE, 1782–1799.
- [128] Ye Li, Richard West, Zhuoqun Cheng, and Eric Missimer. 2014. Predictable communication and migration in the Quest-V separation kernel. In *IEEE Real-Time Systems Symp.* 272–283.
- [129] Jochen Liedtke. 1995. On microkernel construction. In *15th ACM Symp. on Operating System Principles (SOSP-15)*. Copper Mountain Resort, CO. Retrieved from <http://l4ka.org/publications/>
- [130] Pin Liu, Renyu Yang, Jie Sun, and Xudong Liu. 2019. SysOptic: A fine-grained monitoring system for virtual machines based on PMU. In *IEEE Internat. Conf. on Service-Oriented System Engineering (SOSE)*. 244–2446. DOI : <https://doi.org/10.1109/SOSE.2019.00042>
- [131] S. Lozano, T. Lugo, and J. Carretero. 2023. A comprehensive survey on the use of hypervisors in safety-critical systems. *IEEE Access* (2023), 1–1. DOI : <https://doi.org/10.1109/ACCESS.2023.3264825>
- [132] G. Luan et al. 2018. Towards fault-tolerant task backup and recovery in the seL4 microkernel. In *IEEE 42nd Annual Computer Software and Applications Conf. (COMPSAC)*, Vol. 01. 721–726. DOI : <https://doi.org/10.1109/COMPSAC.2018.00107>
- [133] P. Lucas et al. 2017. VOSYSmonitor, a low latency monitor layer for mixed-criticality systems on ARMv8-A. In *29th Euromicro Conf. on Real-Time Systems (ECRTS)*, Vol. 76. 61–618. DOI : <https://doi.org/10.4230/LIPIcs.ECRTS.2017.6>
- [134] Pierre Lucas, Kevin Chappuis, Benjamin Boutin, Julian Vetter, and Daniel Raho. 2018. VOSYSmonitor, a TrustZone-based Hypervisor for ISO 26262 Mixed-critical System. *23rd Conf. of Open Innovations Association (FRUCT)* (2018), 231–238.
- [135] LYNX Software technologies. 2022. LynxSecure Separation Kernel Hypervisor. Retrieved from <https://www.lynx.com/products/lynxsecure-separation-kernel-hypervisor>. (2022).
- [136] José Martins, João Alves, Jorge Cabral, Adriano Tavares, and Sandro Pinto. 2017. μ RTZVisor: A secure and safe real-time hypervisor. *Electronics* 6, 4 (2017), 93.
- [137] José Martins, A. Tavares, M. Solieri, M. Bertogna, and S. Pinto. 2020. Bao: A lightweight static partitioning hypervisor for modern multi-core embedded systems. In *NG-RES@HiPEAC*.
- [138] L. Masing et al. 2022. XANDAR: Exploiting the X-by-Construction Paradigm in Model-based Development of Safety-critical Systems. In *Design, Automation & Test in Europe Conf. & Exhibition (DATE)*. 1–5. DOI : <https://doi.org/10.23919/DATE54114.2022.9774534>
- [139] Miguel Masmano, Ismael Ripoll, Alfons Crespo, and J Metge. 2009. XtratuM: A hypervisor for safety critical embedded systems. In *11th Real-Time Linux Workshop*. 263–272.
- [140] Miguel Masmano, Ismael Ripoll, Alfons Crespo, and Salvador Peiró. 2010. XtratuM for LEON3: An open source hypervisor for high integrity systems. In *Embedded Real Time Software & Systems, May 2010, Toulouse, France*.
- [141] Alejandro Masrur et al. 2010. VM-based real-time services for automotive control applications. In *IEEE 16th International Conference on Embedded and Real-Time Computing Systems and Applications*. 218–223. DOI : <https://doi.org/10.1109/RTCSA.2010.38>
- [142] Enrico Mezzetti, Leonidas Kosmidis, Jaume Abella, and Francisco J. Cazorla. 2018. High-integrity performance monitoring units in automotive chips forreliable timing V&V. *IEEE Micro* 38, 1 (2018), 56–65.
- [143] Microsoft. 2008. Hyper-V architecture. Retrieved from <https://docs.microsoft.com/en-us/windows-server/administration/performance-tuning/role/hyper-v-server/architecture/>. (2008). [Online; accessed 1-October-2021].
- [144] Eric Missimer, Richard West, and Ye Li. 2014. Distributed real-time fault tolerance on a virtualized multi-core system. *OSPRT* (2014), 17.
- [145] Paolo Modica, Alessandro Biondi, Giorgio Buttazzo, and Anup Patel. 2018. Supporting temporal and spatial isolation in a hypervisor for ARM multicore platforms. In *IEEE Internat. Conf. on Industrial Technology (ICIT)*. 1651–1657. DOI : <https://doi.org/10.1109/ICIT.2018.8352429>

- [146] Roberto Morabito, Jimmy Kjällman, and Miika Komu. 2015. Hypervisors vs. lightweight virtualization: A performance comparison. In *IEEE Internat. Conf. on Cloud Engineering*. IEEE, 386–393.
- [147] Daniel Münch et al. 2013. Hardware-Based I/O Virtualization for Mixed Criticality Real-Time Systems Using PCIe SR-IOV. In *IEEE 16th International Conference on Computational Science and Engineering*. 706–713. DOI : <https://doi.org/10.1109/CSE.2013.109>
- [148] Daniel Münch, Michael Paulitsch, Oliver Hanka, and Andreas Herkersdorf. 2015. SgInt: Safeguarding interrupts for hardware-based I/O virtualization for mixed-criticality embedded real-time systems using non transparent bridges (*Architecture of Computing Systems – ARCS 2015*). Springer International Publishing, 15–27.
- [149] D. Münch, M. Paulitsch, and A. Herkersdorf. 2015. IOMPU: Spatial separation for hardware-based I/O virtualization for mixed-criticality embedded real-time systems using non-transparent bridges. In *IEEE 17th Internat. Conf. on High Performance Computing and Communications (HPCC)*. 1037–1044.
- [150] T. Murray et al. 2013. seL4: From general purpose to a proof of information flow enforcement. In *IEEE Symposium on Security and Privacy*. 415–429. DOI : <https://doi.org/10.1109/SP.2013.3>
- [151] V. Muttillio et al. 2019. HW/SW Co-design framework for mixed-criticality embedded systems considering Xtratum-based SW partitions. In *22nd Euromicro Conf. on Digital System Design (DSD)*. 554–561. DOI : <https://doi.org/10.1109/DSD.2019.00085>
- [152] Vittoriano Muttillio, Luca Tiberi, Luigi Pomante, and Paolo Serri. 2019. Benchmarking analysis and characterization of hypervisors for space multicore systems. *Journal of Aerospace Information Systems* 16, 11 (2019), 500–511.
- [153] D. Münch et al. 2015. MPIOV: Scaling hardware-based I/O virtualization for mixed-criticality embedded real-time systems using non transparent bridges to (Multi-Core) multi-processor systems. In *Design, Automation & Test in Europe Conf. & Exhibition (DATE)*. 579–584.
- [154] Yukihide Niimi, Takayuki Ono, Soichiro Arai, Hideki sugimoto, and Riccardo Mariani. 2013. Virtualization Technology and Using Virtual CPU in the Context of ISO26262: The E-Gas Case Study. In *SAE World Congress & Exhibition*. DOI : <https://doi.org/10.4271/2013-01-0196>
- [155] Ruslan Nikolaev and Godmar Back. 2011. Perfctr-Xen: A framework for performance counter virtualization. *SIGPLAN Not.* 46, 7 (2011), 15–26.
- [156] Andrzej Nowak and Georgios Bitzes. 2014. *The Overhead of Profiling Using PMU Hardware Counters*. Technical Report. CERN OpenLab.
- [157] Jan Nowotsch et al. 2014. Monitoring and WCET analysis in COTS multi-core-SoC-based mixed-criticality systems. In *Design, Automation & Test in Europe Conf. & Exhibition (DATE)*.
- [158] Jan Nowotsch et al. 2014. Multi-core interference-sensitive WCET analysis leveraging runtime resource capacity enforcement. In *Euromicro Conf. on Real-Time Systems ECRTS*.
- [159] NXP. 2014. T2080 Advanced QorIQ Debug and Performance Monitoring Reference Manual. (2014). Rev. A 04/2014.
- [160] NXP. 2016. QorIQ T2080 Reference Manual. (2016). Also supports T2081. Doc. No.: T2080RM. Rev. 3, 11/2016.
- [161] NXP. 2021. Layerscape®QorIQ LX2160A Reference Manual. (2021). Supports LX2120A and LX2080A. Doc. No.: LX2160ARM. Rev. 1, 10/2021.
- [162] A. Oliveira et al. 2018. TZ-VirtIO: Enabling standardized inter-partition communication in a trustzone-assisted hypervisor. In *IEEE 27th Int. Symp. on Industrial Electronics (ISIE)*. 708–713.
- [163] Paul Parkinson. 2017. Update on using multicore processors with a commercial ARINC 653 implementation. In *Aviation Electronics Europe*.
- [164] Paul Parkinson and Harold Glenn Tiedeman. 2019. Experiences of civil certification of multi-core processing systems in commercial and military avionics, integration activities, and analysis. *SAE Internat. Journal of Advances and Current Practices in Mobility* 1, 2 (2019), 419–428.
- [165] Anup Patel, Mai Daftedar, Mohamed Shalan, and M. Watheq El-Kharashi. 2015. Embedded hypervisor xvisor: A comparative analysis. In *23rd Euromicro Int. Conf. on Parallel, Distributed, and Network-Based Processing*. IEEE, 682–691.
- [166] Mathieu Patte, A. Crespo, Marco Zulianello, Vincent Lefftz, M. Masmano, and J. Coronel. 2012. System impact of distributed multicore systems. In *Data Systems In Aerospace (DASIA)*, Vol. 701 SP2012. European Space Agency.
- [167] Jon Perez et al. 2014. A safety certification strategy for IEC-61508 compliant industrial mixed-criticality systems based on multicore partitioning. *Euromicro DSD/SEAA Verona, Italy* (2014).
- [168] Jon Perez et al. 2015. A safety concept for an IEC-61508 compliant fail-safe wind power mixed-criticality system based on multicore and partitioning. In *Reliable Software Technologies – Ada-Europe (Lecture Notes in Computer Science)*, Juan Antonio de la Puente and Tullio Vardanega (Eds.), Vol. 9111. Springer Internat. Publishing, 3–17. DOI : https://doi.org/10.1007/978-3-319-19584-1_1
- [169] Jon Perez-Cerrolaza, Jaume Abella, Leonidas Kosmidis, Alejandro J. Calderon, Francisco J. Cazorla, and Jose Luis Flores. 2022. GPU Devices for Safety-Critical Systems: A Survey. *ACM Comput. Surv.* (2022). DOI : <https://doi.org/10.1145/3549526>

- [170] J. Perez-Cerrolaza, F. J. Cazorla, and J. Abella. 2023. Uncertainty management in dependable and intelligent embedded software. *Computer* 56, 3 (2023), 114–117. DOI : <https://doi.org/10.1109/MC.2023.3235093>
- [171] Jon Perez-Cerrolaza, Roman Obermaisser, Jaume Abella, Francisco J. Cazorla, Kim Grüttner, Irune Agirre, Hamidreza Ahmadian, and Imanol Allende. 2020. Multi-core devices for safety-critical systems: A survey. *ACM Comput. Surv.* 53, 4 (2020). DOI : <https://doi.org/10.1145/3398665>
- [172] S. Pinto et al. 2017. Lightweight multicore virtualization architecture exploiting ARM trustzone. In *43rd Annual Conf. of the IEEE Industrial Electronics Society (IECON)*. 3562–3567. DOI : <https://doi.org/10.1109/IECON.2017.8216603>
- [173] Sandro Pinto, Pedro Machado, Daniel Oliveira, David Cerdeira, and Tiago Gomes. 2021. Self-secured devices: High performance and secure I/O access in TrustZone-based systems. *J. Syst. Archit.* 119 (2021).
- [174] Sandro Pinto, Jorge Pereira, Tiago Gomes, Adriano Tavares, and Jorge Cabral. 2017. Ltzvisor: Trustzone is the key. In *29th Euromicro Conf. on Real-Time Systems (ECRTS)*. Schloss Dagstuhl-Leibniz-Zentrum fuer Informatik.
- [175] S. Pinto, A. Tavares, and S. Montenegro. 2016. Hypervisor for real time space applications. In *The 4S Symp.*
- [176] Sandro Pinto, Adriano Tavares, and Sergio Montenegro. 2016. Space and time partitioning with hardware support for space applications. In *DAta Systems in Aerospace Conference (DASIA)*.
- [177] T. Poggi et al. 2018. A hypervisor architecture for low-power real-time embedded systems. In *21st Euromicro Conf. on Digital System Design (DSD)*. 252–259. DOI : <https://doi.org/10.1109/DSD.2018.00054>
- [178] M. Ponos, N. Lazic, M. Bjelica, T. Andjelic, and M. Manic. 2021. One solution for integrating graphics in vehicle digital cockpit. In *29th Telecommunications Forum (TELFOR)*. 1–3. DOI : <https://doi.org/10.1109/TELFOR52709.2021.9653214>
- [179] Gerald J. Popek and Robert P. Goldberg. 1974. Formal requirements for virtualizable third generation architectures. *Commun. ACM* 17, 7 (1974), 412–421. DOI : <https://doi.org/10.1145/361011.361073>
- [180] Héctor Pérez and J. Javier Gutiérrez. 2017. Handling heterogeneous partitioned systems through ARINC-653 and DDS. *Computer Standards & Interfaces* 50 (2017), 258–268. DOI : <https://doi.org/10.1016/j.csi.2016.10.012>
- [181] H. Pérez, J. J. Gutiérrez, S. Peiró, and A. Crespo. 2017. Distributed architecture for developing mixed-criticality systems in multi-core platforms. *Journal of Systems and Software* 123 (2017), 145–159. DOI : <https://doi.org/10.1016/j.jss.2016.08.088>
- [182] Arun Rajan and Nirmala Devi. 2021. Software-based approach for sharing real-time peripherals in a virtualized automotive microcontroller platform. *SAE Internat. Journal of Connected and Automated Vehicles* 4 (2021). DOI : <https://doi.org/10.4271/12-04-02-0016>
- [183] Arun Kumar Sundar Rajan, Armin Feucht, Lothar Gamer, and Idriz Smaili. 2018. Hypervisor for consolidating real-time automotive control units: Its procedure, implications and hidden pitfalls. *J. Syst. Archit.* 82 (2018), 37–48.
- [184] Ralf Ramsauer, Jan Kiszka, and Wolfgang Mauerer. 2020. A novel software architecture for mixed criticality systems (*Digital Transformation in Semiconductor Manufacturing*). Springer Internat. Publishing, 121–128.
- [185] J. Reeser, T. Jankowski, and G. M. Kemper. 2015. Maintaining HMI and SCADA systems through computer virtualization. *IEEE Trans. on Industry Applications* 51, 3 (2015), 2558–2564. DOI : <https://doi.org/10.1109/TIA.2014.2384132>
- [186] Dominik Reinhardt, Maximilian Güntner, and Simon Obermeir. 2015. Virtualized communication controllers in safety-related automotive embedded systems (*Architecture of Computing Systems (ARCS)*). Springer Internat. Publishing, 173–185.
- [187] Dominik Reinhardt and Gary Morgan. 2014. An embedded hypervisor for safety-relevant automotive E/E-systems. In *9th IEEE Internat. Symp. on Industrial Embedded Systems (SIES)*. IEEE, 189–198.
- [188] RTCA. 2012. RTCA DO-178 B/C. Software Considerations in Airborne Systems and Equipment Certification. (2012).
- [189] John Rushby. 2000. *Partitioning in Avionics Architectures: Requirements, Mechanisms, and Assurance*. Report. SRI International (Computer Science Lab).
- [190] Bruno Sá, José Martins, and Sandro Pinto. 2022. A first look at RISC-V virtualization from an embedded systems perspective. *IEEE Trans. on Computers* 71 (2022), 2177–2190.
- [191] SAE. 1996. ARP4761 - Guidelines and methods for conducting the safety assessment process on civil airborne systems and equipment. (1996).
- [192] J. Sahoo, S. Mohapatra, and R. Lath. 2010. Virtualization: A survey on concepts, taxonomy and associated security issues. In *Second Internat. Conf. on Computer and Network Technology*. 222–226. DOI : <https://doi.org/10.1109/ICCNT.2010.49>
- [193] F. Salles, J. Arlat, and J. C. Fabre. 1997. Can we rely on COTS microkernels for building fault-tolerant systems?. In *6th IEEE Computer Society Workshop on Future Trends of Distributed Computing Systems*. 189–194. DOI : <https://doi.org/10.1109/FTDCS.1997.644723>
- [194] Andrew D. Santangelo and Paul Skentzos. 2016. *A Low Cost, Secure Radio Communications System for UAVs*. American Institute of Aeronautics and Astronautics. DOI : <https://doi.org/doi:10.2514/6.2016-057610.2514/6.2016-0576>
- [195] F. Schade, D. Barton, J. Fleischer, and J. Becker. 2021. Evaluation of a hypervisor-based smart controller for industry 4.0 functions in manufacturing. In *IEEE 7th World Forum on Internet of Things (WF-IoT)*. 680–685. DOI : <https://doi.org/10.1109/WF-IoT51360.2021.9595534>

- [196] Sabine Schmaltz and Andrey Shadrin. 2012. Integrated semantics of intermediate-language C and macro-assembler for pervasive formal verification of operating systems and hypervisors from VerisoftXT (*Verified Software: Theories, Tools, Experiments*). Springer Berlin, 18–33.
- [197] Luis Sepúlveda Rodríguez, Julio C. Chavarro-Porras, John A. Sanabria-Ordoñez, Harold E. Castro, and Jeanna Matthews. 2022. A survey of virtualization technologies: Towards a new taxonomic proposal. *Ingeniería e Investigación* 42 (2022). DOI : <https://doi.org/10.15446/ing.investig.97363>
- [198] Alejandro Serrano-Cases et al. 2021. Leveraging hardware QoS to control contention in the Xilinx Zynq UltraScale+ MPSoC. In *ECRTS (LIPICs)*, Vol. 196. Schloss Dagstuhl - Leibniz-Zentrum für Informatik, 3:1–3:26.
- [199] Yicong Shen et al. 2022. Shyper: An embedded hypervisor applying hierarchical resource isolation strategies for mixed-criticality systems. In *Design, Automation & Test in Europe Conf. & Exhibition (DATE)*. 1287–1292. DOI : <https://doi.org/10.23919/DAT54114.2022.9774664>
- [200] Y. Shen and K. Elphinstone. 2015. Microkernel mechanisms for improving the trustworthiness of commodity hardware. In *11th European Dependable Computing Conf. (EDCC)*. 155–166. DOI : <https://doi.org/10.1109/EDCC.2015.16>
- [201] Weisong Shi and Liangkai Liu. 2021. *Autonomous Driving Landscape*. Springer Internat. Publishing, Cham, 1–18. DOI : https://doi.org/10.1007/978-3-030-81564-6_1
- [202] Junaid Shuja, Abdullah Gani, Kashif Bilal, Atta Ur Rehman Khan, Sajjad A. Madani, Samee U. Khan, and Albert Y. Zomaya. 2016. A survey of mobile device virtualization: Taxonomy and state of the art. *ACM Comput. Surv.* 49, 1 (2016). DOI : <https://doi.org/10.1145/2897164>
- [203] Siemens. 2022. Jailhouse Hypervisor. Retrieved from <https://github.com/siemens/jailhouse>. (2022). [Online; accessed 24-February-2022].
- [204] José Simó, Patricia Balbastre, Juan Francisco Blanes, José-Luis Poza-Luján, and Ana Guasque. 2021. The role of mixed criticality technology in industry 4.0. *Electronics* 10, 3 (2021), 226.
- [205] Soham Sinha et al. 2020. A paravirtualized android for next generation interactive automotive systems. In *21st Internat. Workshop on Mobile Computing Systems and Applications (HotMobile '20)*. 50–55.
- [206] Soham Sinha and Richard West. 2021. Towards an integrated vehicle management system in DriveOS. *ACM Trans. Embed. Comput. Syst.* 20, 5s (2021), Article 82. DOI : <https://doi.org/10.1145/3477013>
- [207] Stefan Gabriel Soriga and Mihai Barbulescu. 2013. A comparison of the performance and scalability of Xen and KVM hypervisors. In *RoEduNet Internat. Conf. 12th Edition: Networking in Education and Research*. IEEE, 1–6.
- [208] B. Sprunt. 2002. The basics of performance-monitoring hardware. *IEEE Micro* 22, 4 (2002), 64–71. DOI : <https://doi.org/10.1109/MM.2002.1028477>
- [209] Neal Stollon. 2011. *Infineon Multicore Debug Solution*. Springer US, Boston, MA, 219–230.
- [210] M. Strobl, M. Kucera, A. Foeldi, T. Waas, N. Balbierer, and C. Hilbert. 2013. Towards automotive virtualization. In *Internat. Conf. on Applied Electronics*. 1–6.
- [211] Arun Kumar Sundar Rajan and M. Nirmala Devi. 2021. *Virtualizing an Automotive State-of-the-Art Microcontroller: Techniques and Its Evaluation*. Springer Internat. Publishing, Cham, 19–36. DOI : https://doi.org/10.1007/978-3-030-59897-6_2
- [212] Sugumar T. N. and N. Ramasamy. 2016. Literature survey and review: Virtualization technologies. *Internat. Journal of Printing, Packaging & Allied Sciences* 4, 2 (2016), 1135–1149.
- [213] Runzhou Tao et al. 2021. Formal verification of a multiprocessor hypervisor on arm relaxed memory hardware. In *Proc. of the ACM SIGOPS 28th Symp. on Operating Systems Principles (SOSP '21)*. New York, USA, 866–881. DOI : <https://doi.org/10.1145/3477132.3483560>
- [214] Nicolae Tapus, Lucian Mogosanu, Mihai Carabas, Razvan Deaconescu, and Cosmin Dragomir. 2018. μ QC: A property-based testing framework for L4 microkernels. *Internat. Journal of Critical Computer-Based Systems* 8 (2018), 1. DOI : <https://doi.org/10.1504/IJCCBS.2018.10013023>
- [215] A. Tavares et al. 2012. A customizable and ARINC 653 quasi-compliant hypervisor. In *IEEE International Conference on Industrial Technology*. 140–147. DOI : <https://doi.org/10.1109/ICIT.2012.6209928>
- [216] A. Tavares et al. 2012. RodosVisor - an object-oriented and customizable hypervisor: The CPU virtualization. *IFAC Proceedings Volumes* 45, 4 (2012), 200–205. DOI : <https://doi.org/10.3182/20120403-3-DE-3010.00010>
- [217] Lynx Software Technologies. 2022. Introducing the Z-application, bridging the gap between bare metal and RTOS. Retrieved from <https://www.lynx.com/embedded-systems-learning-center/introducing-the-z-application>. (2022).
- [218] A. Tellabi, J. Fehmi, S. Abdelbast, and C. Ruland. 2019. On the analysis of the impact of scheduling plans in safety critical requirements in virtualization. In *Internat. Conf. on Cybersecurity (ICoSec)*. 20–25. DOI : <https://doi.org/10.1109/ICoSec47621.2019.8971126>
- [219] Asmaa Tellabi, Mithil Parekh, Christoph Ruland, and Mostafa Ezziyyani. 2019. A case study of virtualization used in mixed criticality systems (*Advanced Intelligent Systems for Sustainable Development (AI2SD)*). Springer Internat. Publishing, 1–11.

- [220] Sebouh Toumassian, Rico Werner, and Axel Sikora. 2016. Performance measurements for hypervisors on embedded ARM processors. In *Int. Conf. on Advances in Computing, Commun. and Informatics (ICACCI)*. IEEE, 851–855.
- [221] S. Trujillo, A. Crespo, and A. Alonso. 2013. MultiPARTES: Multicore virtualization for mixed-criticality systems. In *Euromicro Conf. on Digital System Design (DSD)*. 260–265. DOI : <https://doi.org/10.1109/DSD.2013.37>
- [222] Salvador Trujillo, Alfons Crespo, Alejandro Alonso, and Jon Pérez. 2014. MultiPARTES: Multi-core partitioning and virtualization for easing the certification of mixed-criticality systems. *Microprocessor and Microsystems* 38, 8 (2014), 921–932. DOI : <https://doi.org/10.1016/j.micpro.2014.09.004>
- [223] S. H. VanderLeest. 2014. Taming interrupts: Deterministic asynchronicity in an ARINC 653 environment. In *IEEE/AIAA 33rd Digital Avionics Systems Conf. (DASC)*. 8A3–1–8A3–11. DOI : <https://doi.org/10.1109/DASC.2014.6979531>
- [224] S. H. VanderLeest. 2016. The open source, formally-proven seL4 microkernel: Considerations for use in avionics. In *IEEE/AIAA 35th Digital Avionics Systems Conf. (DASC)*. 1–9. DOI : <https://doi.org/10.1109/DASC.2016.7778001>
- [225] Steven H. VanderLeest. 2017. Designing a future airborne capability environment (FACE) hypervisor for safety and security. In *IEEE/AIAA 36th Digital Avionics Systems Conf. (DASC)*. IEEE, 1–9.
- [226] S. H. VanderLeest. 2018. Is formal proof of seL4 sufficient for avionics security? *IEEE Aerospace and Electronic Systems Magazine* 33, 2 (2018), 16–21. DOI : <https://doi.org/10.1109/MAES.2018.160217>
- [227] Steven H. VanderLeest and Dagan White. 2015. MPSoC hypervisor: The safe & secure future of avionics. In *IEEE/AIAA 34th Digital Avionics Systems Conf. (DASC)*. IEEE, 6B5–1–6B5–14.
- [228] Manohar Vanga, Andrea Bastoni, Henrik Theiling, and Björn B. Brandenburg. 2017. Supporting low-latency, low-criticality tasks in a certified mixed-criticality OS. In *25th Internat. Conf. on Real-Time Networks and Systems (RTNS)*. 227–236.
- [229] Prashant Varanasi and Gernot Heiser. 2011. Hardware-supported virtualization on ARM. In *Second Asia-Pacific Workshop on Systems*. Association for Computing Machinery, New York, NY, USA, Article 11, 5 pages.
- [230] VMware. 2007. *A Performance Comparison of hypervisors - Revision: 20070201 Item: PS-004-INF-01-002*. Technical Report. VMware Inc.
- [231] M. Weiß et al. 2014. Integrity verification and secure loading of remote binaries for microkernel-based runtime environments. In *IEEE 13th Int. Conf. on Trust, Security and Privacy in Comput. and Commun.* 544–551. DOI : <https://doi.org/10.1109/TrustCom.2014.69>
- [232] N. J. Wessman et al. 2021. De-RISC: The First RISC-V Space-Grade Platform for Safety-Critical Systems. In *IEEE Space Computing Conference (SCC)*. 17–26. DOI : <https://doi.org/10.1109/SCC49971.2021.00010>
- [233] Richard West, Ye Li, Eric Missimer, and Matthew Danish. 2016. A virtualized separation kernel for mixed-criticality systems. *ACM Trans. Comput. Syst.* 34, 3 (June 2016), 1–41.
- [234] Reinhard Wilhelm. 2018. Mixed feelings about mixed criticality (invited paper). In *18th Int. Workshop on Worst-Case Execution Time Analysis (WCET) (OpenAccess Series in Informatics (OASISs))*, Florian Brandner (Ed.), Vol. 63. 1:1–1:9.
- [235] Windriver. 2017. Certification of Avionics Applications on Multi-core Processors: Opportunities and Challenges. Retrieved from <https://events.windriver.com/wrcd01/wrcm/2017/08/Certification-of-Avionics-Applications-on-Multi-core-Processors-Opportunities-and-Challenges-WP.pdf>. (2017).
- [236] Windriver. 2023. Wind River Helix Virtualization platform. Retrieved from https://www.windriver.com/sites/default/files/2023-04/486559-HelixDatasheetUpdate_Mar2023.pdf. (2023).
- [237] C. Wulf, M. Willig, and D. Göhringer. 2021. A survey on hypervisor-based virtualization of embedded reconfigurable systems. In *31st Internat. Conf. on Field-Programmable Logic and Applications (FPL)*. 249–256. DOI : <https://doi.org/10.1109/FPL53798.2021.00047>
- [238] Xen Project. 2022. Xen Paravirtualization Hypervisor. Retrieved from <https://xenproject.org/>. (2022).
- [239] XenProject. 2020. Cache coloring: Interference-free real-time virtualization. Retrieved from <https://xenproject.org/2020/09/03/cache-coloring-interference-free-real-time-virtualization/>. (2020).
- [240] Sisu Xi, Justin Wilson, Chenyang Lu, and Christopher Gill. 2011. RT-Xen: Towards real-time hypervisor scheduling in Xen. In *Ninth ACM Internat. Conf. on Embedded Software (EMSOFT)*. IEEE, 39–48.
- [241] Sisu Xi, Meng Xu, Chenyang Lu, Linh T. X. Phan, Christopher Gill, Oleg Sokolsky, and Insup Lee. 2014. Real-time multi-core virtual machine scheduling in Xen. In *Internat. Conf. on Embedded Software (EMSOFT)*. 1–10. DOI : <https://doi.org/10.1145/2656045.2656061>
- [242] Irune Yarza, Irune Agirre, Imanol Mugarza, and Jon Perez Cerrolaza. 2022. Safety and security collaborative analysis framework for high-performance embedded computing devices. *Microprocessors and Microsystems* 93 (2022), 104572. DOI : <https://doi.org/10.1016/j.micpro.2022.104572>
- [243] F. Zhang, W. Su, T. Wang, and X. Wang. 2015. SAFEBOX: A verified microkernel based on spatial-temporal isolation. In *2nd Int. Conf. on Information Science and Control Engineering*. 451–455. DOI : <https://doi.org/10.1109/ICISCE.2015.105>
- [244] Zhengjun Zhang, Yanqiang Liu, Jiangtao Chen, Zhengwei Qi, Yifeng Zhang, and Huai Liu. 2021. Performance analysis of open-source hypervisors for automotive systems. In *IEEE 27th Int. Conf. on Parallel and Distributed Systems (ICPADS)*. 530–537. DOI : <https://doi.org/10.1109/ICPADS53394.2021.00072>

- [245] Yongwang Zhao, Zhibin Yang, and Dianfu Ma. 2017. A survey on formal specification and verification of separation kernels. *Frontiers of Computer Science* 11, 4 (2017), 585–607.
- [246] Matteo Zini, Giorgiomaria Cicero, Daniel Casini, and Alessandro Biondi. 2022. Profiling and controlling I/O-related memory contention in COTS heterogeneous platforms. *Software: Practice and Experience* 52, 5 (2022), 1095–1113.
- [247] Mikael Åsberg, Nils Forsberg, Thomas Nolte, and Shinpei Kato. 2011. Towards real-time scheduling of virtual machines without kernel modifications. In *Int. Conf. on Emerging Technologies and Factory Automation (ETFA)*. 1–4. DOI: <https://doi.org/10.1109/ETFA.2011.6059185>

Received 27 June 2023; revised 22 August 2024; accepted 29 August 2025